

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
BỘ MÔN PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH



PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH
ASSIGNMENT 04

Giảng viên hướng dẫn	: TRẦN ĐÌNH QUẾ
Họ và tên sinh viên	: NGUYỄN NGỌC HOÀNG NAM
Mã sinh viên	: B23DCCN585
Lớp	: D23CQCN11-B
Nhóm	: 05

Hà Nội – 2026

LỜI MỞ ĐẦU VÀ TÓM TẮT

Báo cáo thực hiện Assignment 04 của học phần Phát triển các hệ thống thông minh: tìm hiểu mạng nơ-ron tích chập, thiết kế CNN cải tiến và cài đặt bằng NumPy từ đầu, PyTorch, TensorFlow. Ba bộ dữ liệu là MNIST, CIFAR-10 và CIFAR-100, được tải từ Kaggle và chia tập theo một quy trình thống nhất.

Trọng tâm của bài là nói ba mức diễn giải: công thức toán học, phép tính trong chương trình và hành vi quan sát được sau huấn luyện. Một ví dụ ảnh nhỏ được tính xuyên suốt từ convolution đến loss, lan truyền ngược và Adam. Phần code minh họa được đặt cạnh giải thích về kích thước tensor, các trạng thái cần lưu và lỗi cài đặt có thể xảy ra.

Bản mở rộng gồm 54 lượt cho hai kiến trúc, ba dataset, ba cách triển khai và ba seed; thêm 18 lượt đánh giá thành phần trên hai bộ CIFAR bằng PyTorch. Tổng số là 72 lượt trong giao thức chính và bổ sung. Bốn lượt thử kiến trúc ban đầu được lưu riêng để công khai quá trình điều chỉnh; chúng không được tính vào tổng 72.

Các bảng tổng hợp sử dụng trung bình và độ lệch chuẩn mẫu của ba seed. Phép so sánh cải tiến ghép các lượt có cùng seed. Báo cáo phân biệt độ biến thiên do quá trình huấn luyện với độ không chắc chắn khi đổi tập dữ liệu, và không coi ba lần chạy là đủ để kết luận một framework luôn tốt hơn framework khác.

Thành phần bài nộp	Vai trò
Báo cáo PDF	Lý thuyết, quyết định thiết kế, code minh họa, kết quả và thảo luận
Notebook Jupyter	Các bước có thể thực thi và đầu ra đã lưu
CSV, JSON, NPZ, checkpoint	Truy vết chỉ số, nhãn dự đoán và trọng số được chọn
README và môi trường	Hướng dẫn tái lập bằng Anaconda trên Windows

Bảng 1. Các thành phần của bài nộp.

Tài liệu chuyên môn tham khảo slide học phần, hướng dẫn CNN và sách của François Chollet [1-3]. Bộ cục tham khảo mẫu Assignment 03 do người học cung cấp. Bài có trợ lý AI hỗ trợ triển khai, kiểm tra và biên soạn; người nộp cần hiểu các phép tính, giải thích được code và tuân thủ quy định học phần.

Kho mã nguồn: https://github.com/namnguyen05-vn/PTHTTM_Assignment-04. Các dataset gốc được quản lý ngoài Git; liên kết tải và checksum có trong bài.

MỤC LỤC (1/2)

CHƯƠNG 1. BÀI TOÁN VÀ PHẠM VI	5
1.2. Câu hỏi nghiên cứu và tiêu chí đánh giá	6
1.3. Quy trình từ dữ liệu đến kết luận	7
1.4. Môi trường và giới hạn so sánh	8
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT CNN	9
2.2. Tensor ảnh và tính cục bộ	10
2.3. Phép convolution nhiều kênh	11
2.4. Ví dụ số học: từ ảnh đến feature map	12
2.5. Stride, padding và vùng tiếp nhận	13
2.6. ReLU và phép lấy mẫu xuống	14
2.7. Flatten, Dense và số tham số	15
2.8. Softmax và cross-entropy	16
2.9. Lan truyền ngược và quy tắc dây chuyền	17
2.10. Khởi tạo và tối ưu bằng Adam	18
2.11. Batch Normalization và dropout	19
2.12. Residual và bối cảnh phát triển CNN	20
CHƯƠNG 3. DỮ LIỆU VÀ TIỀN XỬ LÝ	21
3.2. Khảo sát MNIST	22
3.3. Khảo sát CIFAR-10	23
3.4. Khảo sát CIFAR-100	24
3.5. Kiểm tra chất lượng dữ liệu	25
3.6. Chia train, validation và test	26
3.7. Chuẩn hóa và tạo minibatch	27
3.8. Lý do chọn dữ liệu và giới hạn phạm vi	28
CHƯƠNG 4. THIẾT KẾ CNN	29
4.2. CNN cải tiến và vị trí can thiệp	30
4.3. Theo dõi shape và đếm tham số	31
4.4. Lý do lựa chọn siêu tham số	32
4.5. Bảo đảm ba backend tương ứng	33
4.6. Giao thức nhiều seed và ablation	34
CHƯƠNG 5. CNN TỪ ĐẦU BẰNG NUMPY	35
5.2. Convolution forward bằng im2col	36
5.3. Convolution backward và col2im	37
5.4. ReLU, MaxPool và Flatten	38
5.5. Dense và loss ổn định số học	39
5.6. BatchNorm forward và backward	40
5.7. Residual và dropout thủ công	41

MỤC LỤC (2/2)

5.8. Cài đặt Adam và trạng thái optimizer	42
5.9. Vòng lặp huấn luyện và chọn checkpoint	43
5.10. Kiểm chứng bản từ đầu	44
CHƯƠNG 6. CÀI ĐẶT BẰNG PYTORCH	45
6.2. Khởi tạo chung và BatchNorm tùy chỉnh	46
6.3. Một bước huấn luyện PyTorch	47
6.4. Eval, lưu và khôi phục checkpoint	48
6.5. Cài đặt các phép ablation	49
6.6. Kiểm chứng PyTorch và quản lý thiết bị	50
CHƯƠNG 7. CÀI ĐẶT BẰNG TENSORFLOW	51
7.2. Chuyển trọng số và thứ tự Flatten	52
7.3. GradientTape và bước cập nhật	53
7.4. BN, dropout và thực thi graph	54
7.5. GPU Windows và checkpoint TensorFlow	55
7.6. Đối chiếu TensorFlow và ba cách cài đặt	56
CHƯƠNG 8. KẾT QUẢ VÀ THẢO LUẬN	57
8.2. Kết quả nhiều seed trên MNIST	58
8.3. Phân tích lỗi MNIST	59
8.4. Kết quả nhiều seed trên CIFAR-10	60
8.5. Phân tích lỗi CIFAR-10	61
8.6. Kết quả nhiều seed trên CIFAR-100	62
8.7. Phân tích lỗi CIFAR-100	63
8.8. Chênh lệch ghép seed và độ ổn định	64
8.9. Đánh giá thành phần trên CIFAR-10	65
8.10. Đánh giá thành phần trên CIFAR-100	66
8.11. Pilot và quá trình điều chỉnh kiến trúc	67
8.12. Chi phí và kiểm chứng sản phẩm	68
8.13. Hạn chế và độ tin cậy	69
CHƯƠNG 9. KẾT LUẬN	70
9.2. Hướng phát triển và bài học thực hành	71
TÀI LIỆU THAM KHẢO (1/2)	72
TÀI LIỆU THAM KHẢO (2/2)	73
PHỤ LỤC A. CHẠY BẰNG ANACONDA	74
PHỤ LỤC B. NOTEBOOK VÀ TỔ CHỨC ĐẦU RA	75
PHỤ LỤC C. ACCURACY CỦA TỪNG SEED	76
PHỤ LỤC D. ABLATION VÀ KIỂM TRA TÁI LẬP	77

DANH MỤC HÌNH VÀ BẢNG

Số trong ngoặc là trang in, không tính bìa. Tên rút gọn; chú thích đầy đủ nằm dưới hình/bảng.

Hình và bảng

Hình 1. Ảnh MNIST và tám đáp ứng Conv1 + ReLU... (9)
 Hình 2. Ảnh MNIST lấy từ phần train; hai ví dụ... (22)
 Hình 3. Số mẫu train của các chữ số sau chia... (22)
 Hình 4. Ảnh CIFAR-10 từ phần train theo tên lớp... (23)
 Hình 5. Phân bố train CIFAR-10 cân bằng giữa... (23)
 Hình 6. Một số lớp CIFAR-100; hình minh họa... (24)
 Hình 7. Phân bố train CIFAR-100 theo 100 nhãn fine. (24)
 Hình 8. Validation MNIST: đường trung bình và... (58)
 Hình 9. Hàng trên: được sửa; hàng dưới: sai... (59)
 Hình 10. Validation CIFAR-10: đường trung bình... (60)
 Hình 11. Hàng trên: được sửa; hàng dưới: sai... (61)
 Hình 12. Validation CIFAR-100: đường trung bình... (62)
 Hình 13. Gộp nhãn fine thành 20 nhóm coarse sau... (63)
 Hình 14. Accuracy và SD của full cùng ba biến... (65)
 Hình 15. Accuracy và SD của full cùng ba biến... (66)
 Hình 16. Pilot thiếu BN Dense và phiên bản cuối... (67)
 Bảng 1. Các thành phần của bài nộp. (1)
 Bảng 2. Đối chiếu yêu cầu Assignment 04. (5)
 Bảng 3. Câu hỏi nghiên cứu và bằng chứng tương ứng. (6)
 Bảng 4. Luồng xử lý và các đầu ra trung gian. (7)
 Bảng 5. Điều kiện phần cứng và phần mềm. (8)
 Bảng 6. Ký hiệu tensor được dùng nhất quán. (10)
 Bảng 7. Convolution đầu tiên trên hai dạng ảnh. (11)
 Bảng 8. Dữ liệu của ví dụ xuyên suốt. (12)
 Bảng 9. Feature map tính bằng NumPy và đối... (12)
 Bảng 10. Một số cấu hình hình học; chỉ dòng đầu... (13)
 Bảng 11. Vùng tiếp nhận của nhánh convolution... (13)
 Bảng 12. Đầu ra ReLU, pooling và Flatten của ví... (14)
 Bảng 13. Chi phí của phần Dense trong mạng nhỏ. (15)
 Bảng 14. Loss và gradient của ví dụ xuyên suốt. (16)

Bảng (tiếp)

Bảng 15. Trạng thái cục bộ phục vụ lan truyền ngược. (17)
 Bảng 16. Cấu hình tối ưu được giữ cố định. (18)
 Bảng 17. Bối cảnh kiến trúc; các mạng tham khảo... (20)
 Bảng 18. Quy mô trước khi tách validation. (21)
 Bảng 19. Kết quả kiểm tra pixel, nhãn và ảnh... (25)
 Bảng 20. Tập chia cố định được dùng cho mọi seed... (26)
 Bảng 21. Khác biệt train, validation và test... (27)
 Bảng 22. Vai trò khác nhau của ba dataset. (28)
 Bảng 23. Kiến trúc baseline theo thứ tự forward. (29)
 Bảng 24. Các thay đổi được kết hợp trong improved. (30)
 Bảng 25. Shape NCHW xuyên suốt hai kiến trúc. (31)
 Bảng 26. Tổng tham số trainable được kiểm tra... (31)
 Bảng 27. Các quyết định thiết kế và giới hạn... (32)
 Bảng 28. Những khác biệt biểu diễn cần được... (33)
 Bảng 29. Số lượt và phạm vi của bản mở rộng. (34)
 Bảng 30. Định nghĩa chính xác của ba phép can thiệp. (34)
 Bảng 31. Tham số, state và cache của các lớp NumPy. (35)
 Bảng 32. Shape trong phép nhân ma trận convolution. (36)
 Bảng 33. Gradient kernel 2×2 từ ví dụ xuyên suốt. (37)
 Bảng 34. Các trường hợp biên của pooling. (38)
 Bảng 35. Gradient ma trận Dense 4×3 trong ví dụ... (39)
 Bảng 36. Liên hệ thuật toán thủ công với định... (41)
 Bảng 37. Bước Adam đầu tiên với learning rate... (42)
 Bảng 38. Các đầu ra nổi quá trình học với báo cáo. (43)
 Bảng 39. Phạm vi diễn giải của từng kiểm tra. (44)
 Bảng 40. Ánh xạ trách nhiệm từ NumPy sang PyTorch. (45)
 Bảng 41. Các thao tác cần có trong bước học. (47)
 Bảng 42. Những dữ liệu cần phối hợp khi khô... (48)
 Bảng 43. Sáu kiểm tra dành riêng cho thiết kế... (49)
 Bảng 44. Đối chiếu PyTorch với NumPy trên batch... (50)

Bảng (tiếp)

Bảng 45. Ánh xạ kiến trúc sang Keras. (51)
 Bảng 46. Bốn phép đối chiếu layout quan trọng. (52)
 Bảng 47. Các điểm kiểm soát của GradientTape. (53)
 Bảng 48. Hành vi TensorFlow theo chế độ và shape. (54)
 Bảng 49. Các yếu tố để tái sử dụng checkpoint... (55)
 Bảng 50. Đối chiếu TensorFlow với NumPy trên... (56)
 Bảng 51. Bài học kỹ thuật từ ba cách triển khai. (56)
 Bảng 52. Phân nhóm kết quả; pilot không tham gia... (57)
 Bảng 53. MNIST: trung bình ± SD của ba seed,... (58)
 Bảng 54. Phân hoạch 10.000 ảnh MNIST theo hai... (59)
 Bảng 55. CIFAR-10: trung bình ± SD của ba seed,... (60)
 Bảng 56. Phân hoạch 10.000 ảnh CIFAR-10 theo hai... (61)
 Bảng 57. CIFAR-100: trung bình ± SD của ba seed,... (62)
 Bảng 58. Phân hoạch 10.000 ảnh CIFAR-100 theo... (63)
 Bảng 59. Bốn lớp recall thấp trong model minh... (63)
 Bảng 60. Accuracy improved trừ baseline theo... (64)
 Bảng 61. PyTorch: trung bình ba seed; delta =... (65)
 Bảng 62. PyTorch: trung bình ba seed; delta =... (66)
 Bảng 63. Các giá trị train/validation của thử... (67)
 Bảng 64. Thời gian train trung bình của ba seed;... (68)
 Bảng 65. Kiểm chứng dữ liệu kết quả, checkpoint... (68)
 Bảng 66. CIFAR-100: độ nhạy trung bình ba seed... (69)
 Bảng 67. Tổng hợp câu trả lời theo bằng chứng... (70)
 Bảng 68. Các hướng chưa thực nghiệm và mục đích... (71)
 Bảng 69. Thứ tự đọc tám notebook trong bài nộp. (75)
 Bảng 70. Cấu trúc thư mục phân biệt nhóm thí nghiệm. (75)
 Bảng 71. 54 lượt chính; accuracy của checkpoint... (76)
 Bảng 72. 18 lượt ablation; không đưa pilot vào... (77)

CHƯƠNG 1. BÀI TOÁN VÀ PHẠM VI

1.1. Chuyển yêu cầu thành các việc có thể kiểm tra

Ảnh yêu cầu Assignment 04 nêu ba dataset từ Kaggle, gồm MNIST và hai tập “big”; tiếp theo là Discover CNN, Improved CNN, cài đặt từ đầu, PyTorch và TensorFlow. Bài diễn giải thành một hệ thống thực nghiệm phân loại ảnh: đầu vào là tensor pixel và đầu ra là nhãn thuộc một tập lớp hữu hạn. Mỗi cách cài đặt phải sử dụng cùng định nghĩa dữ liệu và cùng kiến trúc để việc đối chiếu có ý nghĩa.

Yêu cầu	Cách thực hiện	Minh chứng
Discover CNN	Giải thích phép hợp thành, convolution, loss, gradient	Chương 2; notebook 00
Improved CNN	Thêm BN, residual và dropout; đo tác động thành phần	Chương 4, 8
From scratch	Tự viết forward, backward và Adam bằng NumPy	Chương 5; notebook 02
PyTorch	Tensor và autograd; cùng trọng số khởi tạo	Chương 6; notebook 03
TensorFlow	Keras và GradientTape; đối chiếu layout	Chương 7; notebook 04
Đầu ra hoàn chỉnh	Chạy, lưu và tính lại kết quả; hướng dẫn Anaconda	Chương 8; phụ lục

Bảng 2. Đối chiếu yêu cầu Assignment 04.

Từ “big” chưa có ngưỡng định lượng trong ảnh đề. CIFAR-10 và CIFAR-100 được chọn vì là hai bộ ảnh màu nhiều mẫu, khó hơn chữ số nền đơn giản, đồng thời có thể huấn luyện bản NumPy trên máy cá nhân. Đây là cách hiểu thực hành cần được giảng viên chấp nhận, không phải một khẳng định rằng CIFAR tương đương quy mô ImageNet.

Ba dataset dạng bảng/văn bản chuẩn bị trước được để ngoài phạm vi đã thống nhất. Nếu cần bổ sung, chúng phải có quy trình dữ liệu và giả định mô hình riêng. Không chuyển tùy tiện các cột bảng thành ảnh để làm cho đầu vào giống một bài toán Conv2D.

Sản phẩm cần đạt

Bài vừa cần mô hình học được, vừa cần giải thích vì sao code đúng. Accuracy cao không tự chứng minh gradient được cài đặt đúng; ngược lại, vượt qua một kiểm tra gradient nhỏ chưa bảo đảm mô hình tổng quát tốt trên ảnh mới. Hai loại minh chứng này được trình bày độc lập và bổ trợ nhau.

1.2. Câu hỏi nghiên cứu và tiêu chí đánh giá

Mỗi phần thực nghiệm được gắn với một câu hỏi cụ thể. Cách đặt câu hỏi giúp giới hạn kết luận: kết quả của một mạng nhỏ, một ngân sách epoch và một tập chia cố định không được mở rộng thành nhận định cho mọi CNN hoặc mọi ứng dụng.

Câu hỏi	Đối chiếu cần thực hiện	Tiêu chí đọc kết quả
Q1. NumPy có triển khai đúng không?	Sai phân hữu hạn; shape; ổn định loss; bài toán nhỏ	Sai số trong dung sai và loss có thể giảm
Q2. Ba backend có cùng phép tính không?	Chung trọng số; tắt dropout; so logits và gradient	Sai số số học nhỏ trên các trường hợp kiểm tra
Q3. Improved có ổn định hơn baseline không?	Cùng seed, cùng split, cùng ngân sách	Trung bình chênh lệch và dấu ở từng seed
Q4. Mỗi thành phần đóng vai trò gì?	Bỏ một thành phần khỏi improved	Chênh lệch có điều kiện trên hai bộ CIFAR
Q5. Các lỗi còn lại tập trung ở đâu?	Confusion, recall từng lớp, ảnh được sửa/sai thêm	Mẫu lỗi và giới hạn của diễn giải

Bảng 3. Câu hỏi nghiên cứu và bằng chứng tương ứng.

Accuracy đo tỷ lệ dự đoán đúng. Macro-F1 xem mỗi lớp với trọng số ngang nhau; top-5 kiểm tra nhân thật có nằm trong năm ứng viên mạnh nhất hay không. Cross-entropy phản ánh cả xác suất gán cho nhân thật, nên có thể thay đổi ngay cả khi nhãn argmax chưa đổi. Các chỉ số trả lời những câu hỏi khác nhau và cần được đọc cùng nhau.

Phân tích nhiều seed sử dụng 42, 7 và 2026. Seed của tập chia vẫn là 42, còn seed huấn luyện thay đổi khởi tạo và thứ tự minibatch, đồng thời ảnh hưởng dropout. Do đó độ lệch chuẩn trong bài mô tả biến thiên của huấn luyện trên cùng split, không mô tả biến thiên khi lấy một tập mẫu hoàn toàn khác.

Các quyết định về epoch và checkpoint dựa trên validation. Test đã được xem ở phiên bản trước và được công khai trong quá trình phát triển, vì vậy nghiên cứu mở rộng mang tính khám phá. Việc chạy thêm seed làm kết luận ổn định hơn nhưng không biến tập test đã quan sát thành một holdout mới.

1.3. Quy trình từ dữ liệu đến kết luận

Bước	Đầu vào	Đầu ra và điều kiện chuyển bước
1. Chuẩn bị	ZIP Kaggle, nhãn và metadata	Mảng uint8; tên lớp; checksum
2. Chia tập	Training gốc và seed chia	Train/validation không giao chỉ số; test gốc giữ nguyên
3. Kiểm chứng	Tensor nhỏ và trọng số chung	Gradient, logits, shape vượt qua kiểm tra
4. Huấn luyện	Minibatch của train	History và checkpoint thấp nhất theo validation loss
5. Đánh giá	Checkpoint đã chọn	Logits test, nhãn dự đoán và metrics
6. Tổng hợp	Các lượt hợp lệ trong giao thức	Trung bình, SD, chênh lệch ghép seed, hình và bảng
7. Biên soạn	Kết quả có thể truy vết	Báo cáo, notebook, README và gói bài nộp

Bảng 4. Luồng xử lý và các đầu ra trung gian.

Một epoch đi qua toàn bộ phần train đã chia. Sau epoch, mạng chuyển sang chế độ đánh giá để tính validation. Khi loss tốt hơn các epoch trước, chương trình lưu trạng thái suy luận; khi hết ngân sách, trạng thái tốt nhất được khôi phục trước khi chạy test. Vì vậy kết quả test không mặc nhiên thuộc epoch cuối.

Chuẩn hóa pixel bằng phép chia 255 không cần ước lượng từ dữ liệu. Nếu sau này dùng chuẩn hóa theo mean/std, các thống kê đó phải được học từ train và áp dụng nguyên vẹn cho validation/test. Mọi phép biến đổi ngẫu nhiên cũng cần được giới hạn ở train, trừ khi có một giao thức đánh giá khác được nêu rõ.

Tái lập và khả năng kiểm tra

Chia sẻ mã nguồn chỉ là một phần của tái lập. Người đọc còn cần phiên bản thư viện, định nghĩa tập chia, seed, siêu tham số, quy tắc chọn checkpoint và đầu ra gốc. Bài lưu các yếu tố này để người khác có thể lần từ một ô trong bảng PDF đến đúng thư mục thí nghiệm.

Những lần chạy được tổ chức theo seed và loại thí nghiệm. Lượt seed 42 ban đầu được tái sử dụng khi cấu hình khớp; hai seed mới và các biến thể ablation có thư mục riêng. Cơ chế khóa thư mục tránh hai notebook đồng thời ghi vào cùng một lượt.

1.4. Môi trường và giới hạn so sánh

Thành phần	Thiết lập sử dụng
Máy thực nghiệm	Intel Core i5-12500H; RAM 16 GB; RTX 3050 Laptop 4 GB
Hệ điều hành / môi trường	Windows; Python 3.10 dựa trên Anaconda; Jupyter
Tính toán NumPy	NumPy 1.23.5; forward/backward thủ công trên CPU
PyTorch	2.5.1+cu121; dùng CUDA khi có GPU
TensorFlow	2.10.1; CUDA 11.2, cuDNN 8.1 trên Windows bản địa
Môi trường notebook	Kernel Python (Assignment 04)
Kiểm soát thư viện	Phiên bản thực tế trong runtime_versions.json; môi trường mới trong environment.yml

Bảng 5. Điều kiện phần cứng và phần mềm.

TensorFlow 2.10 được chọn do giới hạn hỗ trợ GPU trực tiếp trên Windows bản địa của các phiên bản sau [12]. PyTorch dùng bộ thư viện CUDA riêng theo gói cài đặt. Hai framework được chạy trong các tiến trình tách biệt để tránh xung đột thư viện và trạng thái GPU. Các lựa chọn phiên bản phục vụ tái lập bài, không phải khuyến nghị về phiên bản mới nhất.

Bản NumPy dùng nhân ma trận đã được tối ưu bên dưới thư viện, nhưng các công thức gradient vẫn do source của bài xây dựng. Cách này đáp ứng mục tiêu from scratch ở mức thuật toán mạng nơ-ron: không sử dụng autograd hoặc lớp CNN có sẵn để thay thế phần lan truyền ngược.

Một số hàng đợi CPU được chạy đồng thời với quá trình GPU và công việc tạo báo cáo. Lịch chạy, số luồng BLAS, khởi tạo GPU và biên dịch TensorFlow đều có thể ảnh hưởng thời gian. Các số giây được lưu để truy vết chi phí; chúng không đủ điều kiện làm benchmark tốc độ giữa ba backend.

Những yếu tố giữ cố định

Cả ba backend dùng cùng tensor đầu vào, cùng split, cùng trọng số khởi tạo NumPy cho mỗi seed, cùng batch size và số epoch. Không yêu cầu kết quả bitwise giống nhau: thứ tự cộng số thực, kernel GPU, dropout và chi tiết triển khai Adam có thể dẫn đến quỹ đạo tối ưu khác nhau.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT CNN

2.1. Học biểu diễn và hợp thành hàm

Trong phân loại ảnh, một cách truyền thống là tự xây dựng đặc trưng rồi học bộ phân loại. Mạng học sâu đưa cả phần biến đổi đặc trưng vào quá trình tối ưu: mỗi tầng nhận biểu diễn từ tầng trước, áp dụng phép biến đổi có tham số và truyền kết quả đi tiếp. Nhãn chỉ xuất hiện ở hàm mục tiêu, nhưng gradient từ hàm mục tiêu có thể cập nhật cả những tầng ở gần ảnh đầu vào [2,3].

$$z = f_L(f_{L-1}(\dots f_2(f_1(x; \theta_1); \theta_2) \dots); \theta_L)$$

Trong biểu thức trên, x là ảnh hoặc một minibatch; θ của mỗi tầng chứa trọng số và bias tương ứng. Các hàm không có tham số như ReLU, pooling hay reshape vẫn là một phần của chuỗi. Góc nhìn hợp thành giải thích vì sao giao diện forward/backward theo từng lớp là một cách cài đặt tự nhiên.

Vì sao cần phi tuyến?

$$h = W_1x + b_1, \quad z = W_2h + b_2$$

$$z = (W_2W_1)x + (W_2b_1 + b_2)$$

Hai phép affine liên tiếp có thể gộp thành một phép affine khác. Do đó, chỉ xếp nhiều lớp tuyến tính không tạo ra lớp hàm biểu diễn phong phú như khi chèn phi tuyến. ReLU làm phép hợp thành phụ thuộc vào những vùng đầu vào khác nhau, giúp mạng xây dựng các ranh giới phân loại phức tạp hơn.

Học biểu diễn không đồng nghĩa mọi kênh đều mang một nhãn ngữ nghĩa rõ ràng. Một feature map chỉ cho biết đáp ứng của bộ lọc ở các vị trí; việc đặt tên “bộ lọc mắt” hoặc “bộ lọc bánh xe” cần thêm bằng chứng. Trong bài, hình đặc trưng được dùng để minh họa phép biến đổi, còn chất lượng được đo bằng dữ liệu đánh giá.



Hình 1. Ảnh MNIST và tám đáp ứng Conv1 + ReLU của NumPy baseline seed 42; từng map dùng thang màu riêng.

2.2. Tensor ảnh và tính cục bộ

Một ảnh số là một mảng giá trị có cấu trúc không gian. Pixel lân cận thường có quan hệ về cạnh, vùng màu hoặc kết cấu. CNN khai thác cấu trúc này bằng cửa sổ cục bộ: mỗi đơn vị đầu ra nhìn một vùng nhỏ, thay vì kết nối độc lập tới mọi pixel của toàn bộ ảnh.

Ký hiệu	Ý nghĩa	Ví dụ CIFAR minibatch
N	Số ảnh trong batch	128
C	Số kênh	3
H, W	Chiều cao và chiều rộng	32, 32
NCHW	Thứ tự NumPy / PyTorch trong bài	$128 \times 3 \times 32 \times 32$
NHWC	Thứ tự đầu vào Keras trong bài	$128 \times 32 \times 32 \times 3$

Bảng 6. Ký hiệu tensor được dùng nhất quán.

Đổi layout là hoán vị trục, không phải thay đổi nội dung ảnh. Chẳng hạn transpose từ NCHW sang NHWC phải chuyển đúng trục kênh. Reshape đơn thuần giữ thứ tự bộ nhớ và có thể trộn giá trị của vị trí hoặc kênh khác nhau, tạo đầu vào sai dù kích thước vẫn phù hợp với model.

```
x_nhwc = x_nchw.transpose(0, 2, 3, 1)
x_back = x_nhwc.transpose(0, 3, 1, 2)
np.testing.assert_array_equal(x_nchw, x_back)
```

Mã 1. Kiểm tra một phép hoán vị layout và phép nghịch đảo.

Chia sẻ trọng số

Cùng một kernel được áp dụng ở nhiều vị trí ảnh. Một đặc trưng cục bộ học ở góc trái vì vậy có thể được phát hiện ở vị trí khác mà không cần một bộ trọng số độc lập. Lợi thế này là một giả định cấu trúc của mô hình; nó phù hợp với ảnh hơn là một bảng mà thứ tự cột chỉ do người tạo file lựa chọn.

Chia sẻ kernel tạo tính tương ứng với phép dịch trong những điều kiện nhất định, nhưng không bảo đảm toàn bộ bộ phân loại bất biến với mọi dịch chuyển. Padding ở biên, lấy mẫu xuống bằng pooling và các lớp Dense phụ thuộc vị trí đều có thể thay đổi đáp ứng. Báo cáo vì thế tránh diễn đạt CNN “luôn bất biến” với vị trí.

2.3. Phép convolution nhiều kênh

Trong code và các framework được dùng, thao tác mang tên convolution thực hiện cross-correlation: kernel được nhân trực tiếp với cửa sổ ảnh, không lật hai chiều. Giữ quy ước này giúp công thức, ví dụ và trọng số chuyển giữa backend khớp nhau.

$$Y_{n,o,i,j} = b_o + \sum_{c=0}^{C_{in}-1} \sum_{u=0}^{K-1} \sum_{v=0}^{K-1} W_{o,c,u,v} X_{n,c,i+u-P,j+v-P}$$

Công thức trên dùng stride 1 và dilation 1. Chỉ số o chọn kênh đầu ra; mỗi kênh đầu ra có một kernel cho từng kênh đầu vào. Các tổng trên chiều kênh và không gian tạo một giá trị đầu ra, sau đó cộng bias. Khi có nhiều kernel đầu ra, mạng có nhiều cách phản ứng với cùng vùng ảnh.

Đại lượng	MNIST Conv1	CIFAR Conv1
Đầu vào một ảnh	$1 \times 28 \times 28$	$3 \times 32 \times 32$
Trọng số OIHW	$8 \times 1 \times 3 \times 3$	$8 \times 3 \times 3 \times 3$
Bias	8	8
Đầu ra khi padding 1	$8 \times 28 \times 28$	$8 \times 32 \times 32$
Tham số trainable	80	224

Bảng 7. Convolution đầu tiên trên hai dạng ảnh.

Tăng số kênh đầu ra cho phép học nhiều bộ lọc hơn, nhưng tăng cả tham số lẫn lượng tính toán ở tầng đó và tầng kế tiếp. Tăng kích thước ảnh chủ yếu làm tăng số vị trí áp dụng kernel; số tham số của convolution vẫn giữ nguyên nếu số kênh và kernel không đổi.

$$\#\theta_{conv} = C_{out}(K^2 C_{in} + 1)$$

Bias có thể được giữ hoặc bỏ tùy thiết kế. Trong bài, convolution có bias ở cả baseline và improved để giữ cài đặt tương ứng. Việc sau đó có BatchNorm không khiến bias tự biến mất khỏi tham số model; số lượng trong bảng phải được tính từ đúng kiến trúc thực thi.

2.4. Ví dụ số học: từ ảnh đến feature map

Ví dụ dưới đây dùng ảnh nhân tạo 5×5 , một kernel 2×2 , stride 1, padding 0 và bias 0. Kích thước nhỏ giúp có thể kiểm tra bằng tay. Đây là bài minh họa độc lập, không phải một mẫu MNIST/CIFAR và không được dùng để tính accuracy thực nghiệm.

Ảnh X: mỗi dòng là một hàng	Kernel K
1 0 2 3 1	1 0
4 6 6 8 2	0 -1
3 1 1 0 2	
1 2 2 4 0	
0 1 3 1 2	

Bảng 8. Dữ liệu của ví dụ xuyên suốt.

Tại góc trên trái, cửa sổ gồm $[[1,0],[4,6]]$. Tổng tích là $1 \times 1 + 0 \times 0 + 4 \times 0 + 6 \times (-1) = -5$. Dịch cửa sổ sang phải một ô cho kết quả -6. Áp dụng cùng quy tắc ở tất cả vị trí tạo feature map 4×4 dưới đây.

Hàng	Các phần tử đầu ra
1	-5 -6 -6 1
2	3 5 6 6
3	1 -1 -3 0
4	0 -1 1 2

Bảng 9. Feature map tính bằng NumPy và đối chiếu vòng lặp trực tiếp.

```
out = np.empty((4, 4), dtype=np.float32)
for i in range(4):
    for j in range(4):
        window = image[i:i+2, j:j+2]
        out[i, j] = np.sum(window * kernel)
```

Mã 2. Bản vòng lặp để kiểm tra của phép tổng tích cục bộ.

Kernel minh họa lấy chênh lệch giữa hai vị trí chéo, nên đáp ứng có thể âm hoặc dương. Kernel học trong CNN thật được cập nhật từ dữ liệu; ta không gán thủ công một phép phát hiện cạnh cho tất cả các lớp. File teaching_example.json lưu toàn bộ giá trị và notebook thực thi lại để kiểm tra các con số.

2.5. Stride, padding và vùng tiếp nhận

$$H_{out} = \left\lfloor \frac{H + 2P - D(K - 1) - 1}{S} \right\rfloor + 1$$

S là bước dịch cửa sổ; P là số pixel đệm mỗi bên; D là dilation. Công thức theo chiều rộng tương tự. Padding ảnh hưởng cả kích thước lẫn cách xử lý biên. Zero padding bổ sung giá trị 0 bên ngoài ảnh, nên một cửa sổ ở sát biên có nội dung khác cửa sổ ở giữa ảnh.

Đầu vào H	K / P / S / D	Đầu ra	Diễn giải
32	3 / 1 / 1 / 1	32	Giữ kích thước như Conv trong bài
32	3 / 0 / 1 / 1	30	Không đệm, mất vị trí biên
32	3 / 1 / 2 / 1	16	Lấy mẫu thưa hơn
32	3 / 2 / 1 / 2	32	Kernel hiệu dụng rộng 5

Bảng 10. Một số cấu hình hình học; chỉ dòng đầu dùng cho Conv thực nghiệm.

Vùng tiếp nhận lý thuyết

Vùng tiếp nhận là phần đầu vào có thể ảnh hưởng tới một đơn vị ở tầng sau. Với r là độ rộng vùng tiếp nhận và j là khoảng cách giữa hai đơn vị kề nhau tính trên ảnh gốc, một lớp kernel k và stride s cập nhật r thành $r + (k-1)j$, rồi cập nhật j thành sj. Đây là vùng có thể tác động, không phải bản đồ mức độ tác động đã học.

Sau lớp	r	j
Đầu vào	1	1
Conv 3×3	3	1
Pool 2×2	4	2
Conv 3×3	8	2
Pool 2×2 của baseline	10	4
Conv residual rồi Pool của improved	14	4

Bảng 11. Vùng tiếp nhận của nhánh convolution trước Flatten.

Dense sau Flatten kết hợp mọi vị trí còn lại, nên logit cuối có thể phụ thuộc trên toàn bộ ảnh. Bảng trên mô tả phần trích xuất cục bộ, không khẳng định toàn mạng chỉ nhìn vùng 10×10 hoặc 14×14. Tách hai mức này giúp tránh nhầm giữa một ô feature map và một nhãn phân loại.

2.6. ReLU và phép lấy mẫu xuống

$$\text{ReLU}(x) = \max(0, x)$$

$$\frac{\partial \text{ReLU}}{\partial x} = \mathbf{1}_{x>0}$$

ReLU giữ phần dương và đưa phần âm về 0. Tại đúng 0, hàm không có đạo hàm duy nhất; cài đặt trong bài chọn gradient 0 theo mask $x>0$. Khi kiểm tra bằng sai phân hữu hạn, cần tránh các điểm không trơn này để không nhầm sự khác biệt của quy ước với lỗi công thức.

Max pooling 2×2 lấy giá trị lớn nhất trong mỗi cửa sổ không chồng lấn. Nó giảm mỗi chiều không gian một nửa và không có trọng số học. Backward đưa gradient về vị trí cực đại đã lưu; nếu nhiều phần tử bằng nhau, argmax chọn một vị trí theo quy ước của cài đặt.

Bước trong ví dụ	Kết quả
ReLU của feature map	0 0 0 1 / 3 5 6 6 / 1 0 0 0 / 0 0 1 2
Pool 2×2 , stride 2	5 6 / 1 2
Vector sau Flatten	[5.0, 6.0, 1.0, 2.0]

Bảng 12. Đầu ra ReLU, pooling và Flatten của ví dụ 5×5 .

Vì sao giảm kích thước?

Giảm không gian làm phần xử lý tiếp theo gọn hơn, đặc biệt khi dùng Flatten rồi Dense. Đổi lại, chi tiết vị trí có thể bị mất. Đối tượng nhỏ hoặc khác biệt rất mảnh giữa hai lớp có thể khó phân biệt sau nhiều lần lấy mẫu xuống. Bài chỉ dùng hai pooling để cân bằng kích thước và thông tin trên ảnh 28×28 hoặc 32×32 .

```
mask = x > 0
relu_output = np.maximum(x, 0)
relu_input_gradient = upstream_gradient * mask
```

Mã 3. ReLU lưu mask ở forward và tái sử dụng ở backward.

Max pooling không đồng nghĩa “chọn đặc trưng quan trọng nhất” theo nhãn. Nó chỉ chọn cực đại số học trong cửa sổ. Việc một đáp ứng lớn có hữu ích cho phân loại hay không phụ thuộc quá trình học và cần được đánh giá bằng kết quả cuối.

2.7. Flatten, Dense và số tham số

Flatten chuyển một tensor đặc trưng thành vector nhưng không học trọng số. Thứ tự phần tử rất quan trọng: cùng một tensor khi flatten theo NCHW và NHWC tạo hai vector khác nhau. Nếu Dense giữ nguyên trọng số, thay đổi thứ tự sẽ thay đổi logit. Bài chuyển layout TensorFlow về NCHW trước Flatten để giữ sự tương ứng.

$$Y = XW + b$$

$$\# \theta_{dense} = (D_{in} + 1) D_{out}$$

Lớp phân loại	MNIST	CIFAR
Sau hai pooling	$16 \times 7 \times 7$	$16 \times 8 \times 8$
Độ dài Flatten	784	1.024
Dense 64: tham số	50.240	65.600
Dense cuối 10 lớp	650	650
Dense cuối 100 lớp	Không dùng	6.500

Bảng 13. Chi phí của phần Dense trong mạng nhỏ.

Phần lớn tham số baseline nằm ở Dense 64. Trên CIFAR-10, lớp này chiếm 65.600 trong tổng 67.642 tham số, khoảng 97%. Vì vậy tăng kích thước feature map trước Flatten có thể làm mạng lớn nhanh dù các convolution vẫn nhỏ. Đây là một lý do cần tính kích thước trước khi triển khai.

Một lựa chọn mở rộng: global average pooling

Global average pooling lấy trung bình không gian của từng kênh và tạo vector dài bằng số kênh. Nó có thể giảm mạnh số tham số ở phần phân loại, nhưng cũng thay đổi cách mạng giữ thông tin vị trí. Trong bài, đây là hướng phát triển được phân tích về cấu trúc; chưa được huấn luyện nên không có kết quả so sánh thực nghiệm.

Đếm tham số cần phân biệt tham số tối ưu bằng gradient và trạng thái lưu phục vụ suy luận. Trọng số, bias, gamma và beta của BN là trainable. Running mean/variance được cập nhật từ batch và cần nằm trong checkpoint, nhưng không được cộng vào số tham số trainable.

2.8. Softmax và cross-entropy

$$p_k = \frac{\exp(z_k - m)}{\sum_j \exp(z_j - m)}, \quad m = \max_j z_j$$

$$L = -\frac{1}{N} \sum_{n=1}^N \log p_{n, y_n}$$

Logits là điểm số chưa chuẩn hóa. Softmax biến chúng thành các số không âm có tổng bằng 1. Trừ cùng một m khỏi mọi logit không đổi phân phối, nhưng giúp tránh tràn số khi tính $\exp$. Cross-entropy phạt việc gán xác suất nhỏ cho nhãn thật; sai với độ tự tin cao thường bị phạt mạnh hơn sai với phân phối còn phân tán.

Đại lượng của ví dụ	Giá trị
Vector sau pooling	[5.0, 6.0, 1.0, 2.0]
Nhãn thật	1 trong ba lớp 0, 1, 2
Logits	0,800, -0,500, -0,300
Softmax	0,62290, 0,16976, 0,20734
Cross-entropy	1,773375
Gradient theo logits	0,62290, -0,83024, 0,20734

Bảng 14. Loss và gradient của ví dụ xuyên suốt.

$$\frac{\partial L}{\partial z_{n,k}} = \frac{p_{n,k} - \mathbf{1}_{k=y_n}}{N}$$

Nhãn nguyên được sử dụng thay cho việc lưu one-hot cho toàn bộ dataset. Công thức gradient vẫn tương đương: sao chép xác suất, trừ 1 ở vị trí nhãn thật rồi chia kích thước batch. Nếu loss đã lấy trung bình mà gradient lại chia N thêm một lần, bước cập nhật sẽ nhỏ hơn dự định.

Confidence trong predictions.csv là xác suất softmax lớn nhất. Con số này là đầu ra của model, chưa được hiệu chỉnh xác suất. Nó phù hợp để chọn ví dụ “sai nhưng tự tin” cho phân tích, nhưng không được diễn giải tự động thành xác suất đúng trong mọi tình huống thực tế.

2.9. Lan truyền ngược và quy tắc dây chuyền

Forward tính biểu diễn và loss. Backward nhận gradient từ phía đầu ra rồi tính gradient theo đầu vào và tham số của từng lớp. Các lớp được duyệt theo thứ tự ngược; mỗi lớp chỉ cần biết phép biến đổi cục bộ và các giá trị đã lưu ở forward [2,3].

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial h} \frac{\partial h}{\partial x}$$

$$dW = X^T G, \quad db = \sum_n G_n, \quad dX = G W^T$$

Ba biểu thức ở dòng dưới áp dụng cho Dense $Y=XW+b$ với G là gradient theo Y . Kích thước là một cách kiểm tra nhanh: nếu X có $N \times D_{in}$ và G có $N \times D_{out}$ thì dW phải là $D_{in} \times D_{out}$. Tuy nhiên, đúng kích thước chưa đủ bảo đảm đúng nội dung, nên bài còn kiểm tra bằng sai phân hữu hạn.

Phép toán	Thông tin lưu ở forward	Điểm cần chú ý khi backward
Dense	Đầu vào X	Cộng bias theo batch; không chia batch hai lần
ReLU	Mask $x > 0$	Đạo hàm tại 0 theo quy ước
MaxPool	Argmax mỗi cửa sổ	Đưa gradient về đúng vị trí đã chọn
Conv	Cửa sổ im2col và shape	Cộng dồn đóng góp từ cửa sổ chồng lấn
Residual	Nhánh F và mask ReLU	Cộng gradient từ nhánh F với nhánh identity

Bảng 15. Trạng thái cục bộ phục vụ lan truyền ngược.

Điểm đặc biệt của convolution là cùng trọng số xuất hiện ở nhiều vị trí, nên gradient của trọng số là tổng đóng góp từ tất cả vị trí và tất cả mẫu trong batch. Đồng thời một pixel có thể nằm trong nhiều cửa sổ, nên gradient đầu vào phải cộng dồn. Ghi đề thay vì cộng có thể khiến mạng vẫn chạy nhưng học theo gradient sai.

Gradient numerical dùng xấp xỉ trung tâm $[L(\theta+h)-L(\theta-h)]/(2h)$. h quá lớn làm xấp xỉ kém; h quá nhỏ gây mất chính xác do trừ hai số gần nhau. Bài chọn h và dung sai phù hợp float32, dùng tensor nhỏ và công khai phạm vi phần tử được kiểm tra.

2.10. Khởi tạo và tối ưu bằng Adam

Khởi tạo quyết định điểm bắt đầu của bài toán tối ưu. Nếu mọi neuron đối xứng có cùng trọng số, chúng có thể nhận các cập nhật giống nhau. Trong bài, trọng số được lấy ngẫu nhiên với thang phương sai $2/\text{fan-in}$ cho các tầng dùng ReLU [11]; bias bắt đầu từ 0. Mỗi seed tạo một bộ khởi tạo chung để chuyển sang cả ba backend.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

Adam giữ trung bình động của gradient và bình phương gradient [10]. Hiệu chỉnh ở các bước đầu bù việc các trạng thái bắt đầu bằng 0. Learning rate η điều khiển thang cập nhật; epsilon tránh mẫu số bằng 0 và cũng là một chi tiết có thể được đặt khác vị trí trong các cách triển khai.

Siêu tham số	Giá trị	Vai trò
η	0,001	Thang bước cập nhật
β_1 / β_2	0,9 / 0,999	Tốc độ cập nhật hai trung bình động
ϵ	1e-8	Ổn định mẫu số
Batch size	128	Số mẫu mỗi bước; batch cuối có thể nhỏ hơn

Bảng 16. Cấu hình tối ưu được giữ cố định.

Bài dùng cùng ngân sách 5 epoch cho MNIST, 10 cho CIFAR-10 và 12 cho CIFAR-100. Đây là ngân sách thực hành phù hợp mạng nhỏ và bản NumPy, chưa phải kết quả tìm kiếm siêu tham số. Nếu kéo dài epoch hoặc thay lịch learning rate, cần xem đó là một giao thức mới và so sánh trong cùng điều kiện.

2.11. Batch Normalization và dropout

$$\mu = \frac{1}{M} \sum_i x_i, \quad \sigma^2 = \frac{1}{M} \sum_i (x_i - \mu)^2$$

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

BN chuẩn hóa một nhóm giá trị rồi học scale γ và shift β [7]. Với tensor ảnh, nhóm thống kê là N,H,W riêng từng kênh; với Dense, nhóm là chiều batch riêng từng đặc trưng. Train dùng thống kê batch, còn eval dùng running mean/variance đã tích lũy. Vì vậy cùng đầu vào nhưng khác chế độ có thể tạo đầu ra khác nhau.

$$\mu_{run} \leftarrow 0.9\mu_{run} + 0.1\mu_{batch}$$

Bài dùng phương sai tổng thể ở cả ba cách cài đặt. Lớp BN tùy chỉnh trong PyTorch phục vụ đối chiếu với NumPy và Keras; nó không được gọi là hành vi mặc định của nn.BatchNorm. Công thức running được ghi trực tiếp vì tham số momentum có cách diễn đạt khác nhau giữa các thư viện.

Inverted dropout

$$y = \frac{m \odot x}{1-p}, \quad m_i \sim \text{Bernoulli}(1-p)$$

Ở train, dropout che ngẫu nhiên một phần đặc trưng rồi chia phần giữ lại cho $1-p$ để bảo toàn kỳ vọng [9]. Ở eval, dropout trở thành identity. Trong bài $p=0,25$ và chỉ dùng ở phần phân loại sau Dense, BN và ReLU. Dropout không bổ sung tham số học nhưng làm đường forward lúc train ngẫu nhiên.

BN và dropout có thể hữu ích, nhưng không có bảo đảm rằng thêm chúng luôn tăng accuracy trong mọi mạng hoặc mọi ngân sách. BN thay đổi tối ưu và trạng thái suy luận; dropout có thể làm học chậm khi mạng nhỏ. Vì vậy bản mở rộng đánh giá từng can thiệp bằng các lượt chạy có kiểm soát, thay vì suy ra tác dụng chỉ từ mô tả lý thuyết.

2.12. Residual và bối cảnh phát triển CNN

$$y = \text{ReLU}(F(x) + x)$$

$$\frac{\partial L}{\partial x} = g + \left(\frac{\partial F}{\partial x}\right)^T g$$

Ở công thức gradient, g là gradient sau khi đi qua ReLU ngoài phép cộng. Nhánh identity tạo một đường truyền trực tiếp, còn F học phần biến đổi bổ sung [8]. Trong model của bài, F gồm Conv 3×3 và BN, giữ nguyên 16 kênh và kích thước không gian nên không cần phép chiếu để cộng hai nhánh.

Phép tắt skip trong ablation chỉ bỏ phần $+x$; convolution và BN bên trong F vẫn giữ nguyên. Nhờ đó số tham số không đổi, giúp diễn giải chênh lệch rõ hơn so với việc xóa toàn bộ block. Kết luận vẫn có điều kiện trên vị trí block, cấu hình BN, optimizer và số epoch đã chọn.

Mốc tham khảo	Ý tưởng liên quan	Liên hệ với bài
LeNet [14]	CNN học từ ảnh chữ viết và nhận dạng tài liệu	Động cơ dùng khối Conv và giảm không gian
AlexNet [15]	Mở rộng CNN và huấn luyện bằng GPU trên ảnh tự nhiên	Vai trò tài nguyên và regularization
VGG [16]	Khảo sát độ sâu với kernel nhỏ 3×3	Thiết kế kernel đồng nhất
ResNet [8]	Học residual bằng đường nối cộng	Block cải tiến và phép tắt skip

Bảng 17. Bối cảnh kiến trúc; các mạng tham khảo không được huấn luyện trong bài.

Mô hình thực nghiệm là một CNN nhỏ được thiết kế cho mục tiêu học thuật; nó không phải bản cải đặt đầy đủ của LeNet, VGG hay ResNet. Các công trình trên cung cấp ý tưởng để giải thích lựa chọn, còn mọi số liệu accuracy trong báo cáo được lấy từ source và dữ liệu của Assignment 04.

Từ bối cảnh này, tiêu chí đánh giá phù hợp là tính đúng của triển khai, tính nhất quán khi so sánh và khả năng lý giải kết quả. Một mạng ít tham số với ngân sách ngắn không nên được quảng bá là mức tốt nhất của benchmark chỉ vì sử dụng một block residual.

CHƯƠNG 3. DỮ LIỆU VÀ TIỀN XỬ LÝ

3.1. Nguồn dữ liệu và quy mô

Ba bộ dữ liệu đều là bài toán phân loại ảnh có nhãn, phù hợp với Conv2D. MNIST cung cấp mức độ khó thấp để kiểm tra quy trình; CIFAR-10 bổ sung màu sắc và ảnh tự nhiên; CIFAR-100 tăng số lớp cần phân biệt trong cùng độ phân giải. Các nguồn Kaggle là bản phân phối tải về của bài; thông tin nguồn gốc CIFAR được đối chiếu với trang của tác giả [4-6,13].

Dataset	Training gốc	Test	Ảnh / số lớp
MNIST	60.000	10.000	28×28×1 / 10
CIFAR-10	50.000	10.000	32×32×3 / 10
CIFAR-100	50.000	10.000	32×32×3 / 100

Bảng 18. Quy mô trước khi tách validation.

MNIST: [oddrational/mnist-in-csv](https://oddrational.com/mnist-in-csv).

CIFAR-10: [pankrzysiu/cifar10-python](https://pankrzysiu.com/cifar10-python).

CIFAR-100: [fedesoriano/cifar100](https://fedesoriano.com/cifar100).

Tính cả training gốc và test, MNIST có 54,88 triệu giá trị kênh-pixel. Mỗi bộ CIFAR có 184,32 triệu giá trị, khoảng 3,36 lần MNIST. Số ảnh CIFAR ít hơn MNIST, nhưng mỗi ảnh có ba kênh và kích thước lớn hơn. Số giá trị pixel chỉ là một chỉ dấu quy mô, không đo đầy đủ độ khó ngữ nghĩa hoặc chi phí học.

Định dạng lưu trữ

MNIST được tải ở dạng CSV có nhãn và các cột pixel. CIFAR được đọc từ các batch Python kèm metadata tên lớp. Bước chuẩn bị chuyển chúng về cùng cấu trúc NPZ: x, y, x_test, y_test, train_ids, val_ids và class_names. Nhãn được giữ kiểu số nguyên và thứ tự lớp từ dữ liệu gốc.

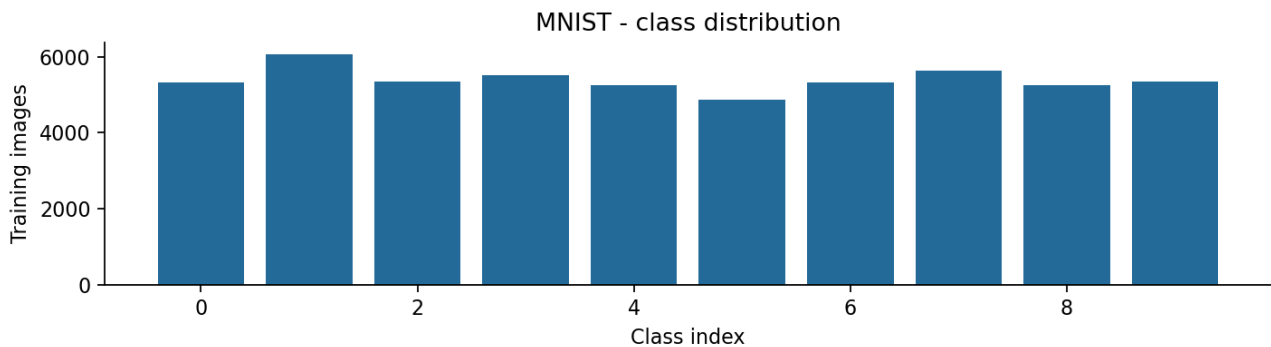
ZIP và mảng đã chuẩn bị có checksum SHA-256 để xác định đúng phiên bản dữ liệu. Git lưu link, checksum và chỉ số chia tập; dữ liệu ảnh lớn được đặt ngoài repository. Cách tổ chức này giảm kích thước bài nộp nhưng vẫn cho phép tải và tái tạo dữ liệu.

3.2. Khảo sát MNIST

MNIST gồm các chữ số viết tay từ 0 đến 9. Ảnh xám 28×28 có nền tương đối đơn giản và vùng chữ được căn chỉnh, giúp một CNN nhỏ học nhanh. Khó khăn còn lại đến từ cách viết khác nhau, nét mờ, nét nổi hoặc hình dạng dễ nhầm. Trong bài, MNIST còn đóng vai trò phát hiện sớm lỗi chuẩn hóa, loss hoặc vòng lặp huấn luyện.



Hình 2. Ảnh MNIST lấy từ phần train; hai ví dụ cho mỗi lớp.



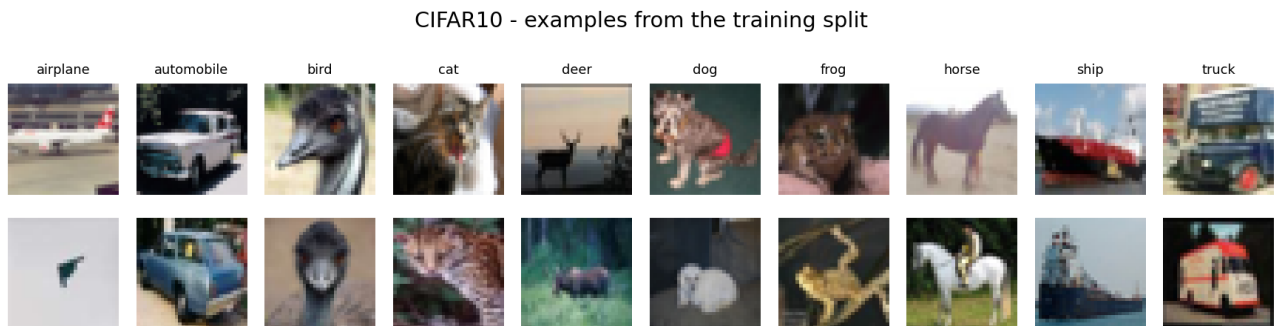
Hình 3. Số mẫu train của các chữ số sau chia validation.

Phần train có 54.001 mẫu và validation có 5.999 mẫu. Đây là kết quả làm tròn 10% riêng từng lớp; tổng vẫn đúng 60.000 mẫu training gốc. Test giữ 10.000 mẫu. Phân bố chữ số không hoàn toàn bằng nhau, vì vậy accuracy và macro-recall không bắt buộc trùng nhau như trên hai bộ CIFAR cân bằng.

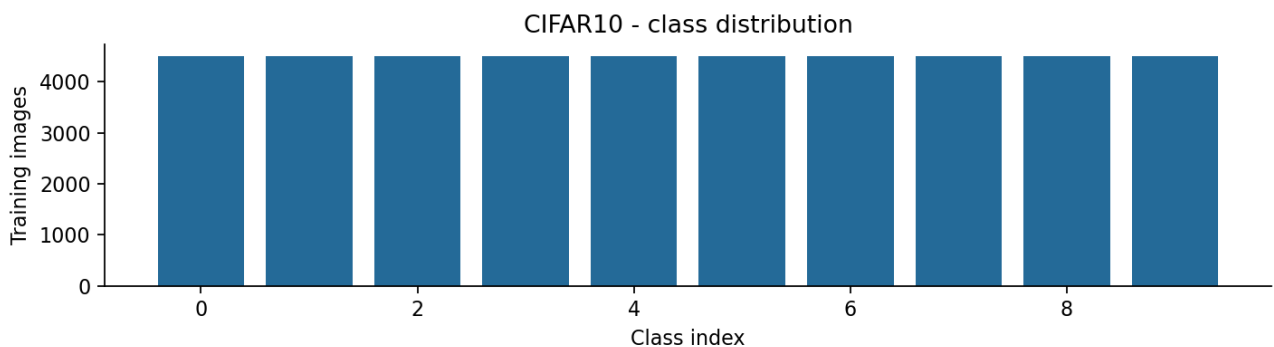
Tham chiếu luôn dự đoán lớp xuất hiện nhiều nhất ở train đạt accuracy test 11,35%. Giá trị này được tính từ đúng dữ liệu của bài, giúp đọc kết quả CNN trong bối cảnh một quy tắc dự đoán tối thiểu.

3.3. Khảo sát CIFAR-10

CIFAR-10 có mười lớp ảnh tự nhiên: airplane, automobile, bird, cat, deer, dog, frog, horse, ship và truck. Mỗi ảnh RGB chỉ có 32×32 pixel. Đối tượng có thể thay đổi tư thế, tỷ lệ, ánh sáng và nền; nhiều chi tiết hữu ích ở ảnh gốc không còn rõ sau khi giảm độ phân giải.



Hình 4. Ảnh CIFAR-10 từ phần train theo tên lớp gốc.



Hình 5. Phân bố train CIFAR-10 cân bằng giữa mười lớp.

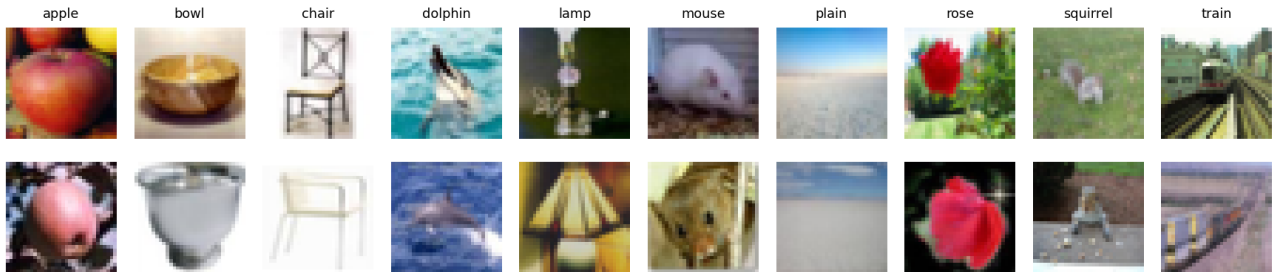
Mỗi lớp có 4.500 mẫu train, 500 validation và 1.000 test. Dữ liệu không cần trọng số lớp để bù một mất cân bằng về số mẫu vốn không hiện diện trong split này. Điều đó không có nghĩa các lớp khó ngang nhau: hình dạng, bối cảnh và mức giống nhau giữa lớp vẫn khác biệt.

Một mô hình luôn đoán một lớp đạt 10% trên test. Vì mỗi lớp test có cùng số mẫu, macro-recall bằng accuracy. Macro-F1 vẫn bổ sung thông tin do còn phụ thuộc precision và phân bố nhãn dự đoán.

3.4. Khảo sát CIFAR-100

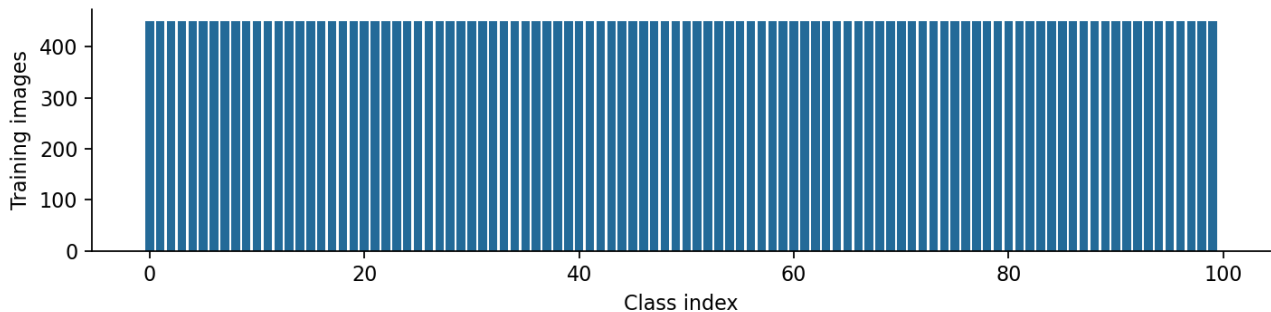
CIFAR-100 giữ kích thước ảnh $32 \times 32 \times 3$ nhưng tăng lên 100 nhãn fine. Mỗi lớp có 500 ảnh training gốc và 100 ảnh test. Sau chia validation, chỉ còn 450 ảnh train cho mỗi lớp, bằng một phần mười CIFAR-10. Mạng phải học nhiều ranh giới phân biệt hơn từ số ảnh trên mỗi lớp nhỏ hơn.

CIFAR100 - examples from the training split



Hình 6. Một số lớp CIFAR-100; hình minh họa không đại diện toàn bộ 100 lớp.

CIFAR100 - class distribution



Hình 7. Phân bố train CIFAR-100 theo 100 nhãn fine.

Metadata còn tổ chức các lớp fine thành 20 nhóm coarse. Huấn luyện và các chỉ số chính của bài đều dùng 100 lớp fine. Ma trận coarse ở phần phân tích lỗi chỉ gộp nhãn dự đoán fine sau đánh giá, không phải một model mới được huấn luyện trên 20 lớp.

Top-5 được đọc bên cạnh accuracy: model có thể đưa nhãn thật vào nhóm năm ứng viên nhưng chưa xếp nó cao nhất. Tham chiếu luôn đoán một lớp chỉ đạt accuracy 1%; con số này giúp thấy độ khó tăng lên dù quy mô toàn bộ ảnh không đổi so với CIFAR-10.

3.5. Kiểm tra chất lượng dữ liệu

Kiểm tra	MNIST	CIFAR-10	CIFAR-100
Kiểu pixel	uint8	uint8	uint8
Miền pixel	0-255	0-255	0-255
Nhãn hợp lệ	Có	Có	Có
Chỉ số train/val giao nhau	Không	Không	Không
Số ảnh training gốc	60.000	50.000	50.000
Ảnh training gốc duy nhất	60.000	50.000	49.986
Hash chung train/validation	0	0	4
Hash chung train/test	0	0	8
Hash chung validation/test	0	0	2

Bảng 19. Kết quả kiểm tra pixel, nhãn và ảnh trùng hoàn toàn.

Không giao nhau theo chỉ số chỉ chứng minh một hàng dữ liệu không bị đưa vào hai phần. Hai hàng khác nhau vẫn có thể chứa cùng ảnh. Vì vậy bài kiểm tra thêm hash pixel. CIFAR-100 có ảnh trùng hoàn toàn và một số nội dung xuất hiện ở cả phần dùng để học hoặc chọn checkpoint với test.

Số hash chung là số nội dung ảnh trùng, không mặc nhiên là số hàng cần bỏ. Trong phép phân tích độ nhảy đã thực hiện cho seed 42, có 10 ảnh test có pixel xuất hiện trong training gốc, nên còn 9.990 ảnh sau loại các mẫu đó. Benchmark chính vẫn giữ split gốc và hạn chế này được công khai.

```
assert x.dtype == np.uint8
assert int(x.min()) >= 0 and int(x.max()) <= 255
assert y.min() >= 0 and y.max() < classes
assert set(train_ids).isdisjoint(set(val_ids))
assert len(train_ids) + len(val_ids) == len(y)
```

Mã 4. Các điều kiện cấu trúc cần thỏa trước huấn luyện.

Hash chính xác không phát hiện ảnh gần giống, ảnh đã crop, đổi màu hoặc nén khác. Việc không tìm thấy hash trùng ở MNIST/CIFAR-10 vì thế không phải bằng chứng rằng mọi dạng rò rỉ ngữ nghĩa đều đã bị loại. Một nghiên cứu nghiêm ngặt hơn cần xem xét nhóm nguồn ảnh trước khi chia.

3.6. Chia train, validation và test

Dataset	Train	Validation	Test
MNIST	54,001	5,999	10,000
CIFAR10	45,000	5,000	10,000
CIFAR100	45,000	5,000	10,000

Bảng 20. Tập chia cố định được dùng cho mọi seed huấn luyện.

Validation được lấy theo từng lớp từ training gốc. Với mỗi lớp, chương trình trộn chỉ số bằng RNG seed 42, lấy khoảng 10% làm validation và giữ phần còn lại cho train. Sau đó hai danh sách chỉ số được trộn và lưu lại. Không có mẫu test nào được đưa vào optimizer.

```
rng = np.random.default_rng(42)
train_ids, val_ids = [], []
for label in range(classes):
    ids = np.flatnonzero(y == label)
    rng.shuffle(ids)
    n_val = round(len(ids) * 0.1)
    val_ids.extend(ids[:n_val])
    train_ids.extend(ids[n_val:])
```

Mã 5. Tách validation có phân tầng; đoạn rút gọn từ src/data.py.

Seed huấn luyện khác seed chia tập. Khi đổi từ 42 sang 7 hoặc 2026 trong vòng lặp học, các chỉ số train/validation không được chia lại. Nếu vừa đổi khởi tạo vừa đổi tập chia, chênh lệch quan sát sẽ gộp hai nguồn biến thiên và không còn trả lời đúng câu hỏi ổn định của quá trình huấn luyện trên cùng dữ liệu.

Vì sao cần validation độc lập?

Loss train tham gia cập nhật trọng số nên thường lạc quan về khả năng tổng quát. Validation cung cấp tín hiệu chọn checkpoint trong ngân sách đã định. Test chỉ dùng để báo cáo chất lượng của checkpoint đó. Dùng accuracy test để chọn epoch sẽ làm test tham gia chọn mô hình, khiến cách diễn giải đánh giá mất độc lập.

Ba seed cùng dùng test 10.000 ảnh, nên ba accuracy không phải ba tập test độc lập. Độ lệch chuẩn giữa seed không thay thế một đánh giá trên dữ liệu từ nguồn mới. Đây là giới hạn cần giữ khi đọc các bảng trung bình $\pm$ SD ở chương kết quả.

3.7. Chuẩn hóa và tạo minibatch

Mảng ảnh lưu uint8 để tiết kiệm bộ nhớ. Chỉ khi lấy một minibatch, chương trình chuyển ảnh sang float32 và chia 255. Các giá trị sau chuẩn hóa nằm trong $[0,1]$. Phép biến đổi giống nhau ở train, validation và test, không dùng bất kỳ nhãn hoặc thống kê test nào.

```
order = ids.copy()
if seed is not None:
    np.random.default_rng(seed).shuffle(order)
for start in range(0, len(order), batch_size):
    ix = order[start:start + batch_size]
    images = x[ix].astype(np.float32) / 255.0
    labels = y[ix]
    yield images, labels
```

Mã 6. Tạo batch từ chỉ số; batch cuối không bị bỏ.

Nếu số mẫu không chia hết cho 128, batch cuối nhỏ hơn. Vòng lặp tính tổng loss theo số mẫu thực tế của từng batch rồi chia tổng số mẫu, tránh việc batch nhỏ có trọng số ngang một batch đầy. Gradient của cross-entropy cũng chia theo đúng kích thước batch hiện tại.

Chế độ	Thứ tự batch	Biến đổi	Model
Train	Shuffle theo seed + epoch	float32 / 255	Dropout bật, BN dùng batch
Validation	Giữ thứ tự chỉ số đã lưu	float32 / 255	Dropout tắt, BN dùng running
Test	Thứ tự test gốc	float32 / 255	Checkpoint tốt nhất, eval

Bảng 21. Khác biệt train, validation và test trong pipeline.

Không dùng data augmentation trong giao thức hiện tại. Điều này giữ dữ liệu đầu vào tương ứng giữa NumPy và framework, đồng thời giúp nghiên cứu tập trung vào kiến trúc. Augmentation là hướng phát triển có tiềm năng, nhưng cần thiết kế riêng và áp dụng cùng điều kiện cho những model được so sánh.

Dữ liệu được tải vào bộ nhớ theo dataset và sử dụng lại qua các epoch trong một lượt. Các framework nhận cùng batch do hàm chung tạo, thay vì mỗi framework tự chia hoặc shuffle bằng một pipeline riêng. Nhờ vậy phần khác biệt ngẫu nhiên do thứ tự minibatch được kiểm soát rõ.

3.8. Lý do chọn dữ liệu và giới hạn phạm vi

Tiêu chí	MNIST	CIFAR-10	CIFAR-100
Cấu trúc	Ảnh xám chữ số	Ảnh RGB tự nhiên	Ảnh RGB tự nhiên
Độ khó mục tiêu	Kiểm tra mạng nhô học được	Phân biệt vật thể và bối cảnh	Phân biệt 100 nhãn fine
Train trên mỗi lớp	Không hoàn toàn đều	4.500	450
Mục đích đối chiếu	Mức cơ sở	Ảnh màu 10 lớp	Nhiều lớp với ít mẫu mỗi lớp

Bảng 22. Vai trò khác nhau của ba dataset.

Chọn ba mức độ khó giúp quan sát một cái tiến có tác dụng khác nhau khi bài toán đơn giản hoặc phức tạp. Trên MNIST, baseline đã có thể đạt accuracy cao nên phần dư để cải thiện nhỏ; trên CIFAR, giới hạn biểu diễn, dữ liệu mỗi lớp và thời gian tối ưu có thể bộc lộ rõ hơn. Đây là giả thuyết để kiểm tra, không phải kết luận được đặt trước.

Đánh giá cách hiểu “2 big”

CIFAR có nhiều ảnh, ba kênh màu và nội dung đa dạng hơn MNIST, phù hợp một bài thực hành yêu cầu cả bản từ đầu. Tuy nhiên hai bộ có cùng 60.000 ảnh và độ phân giải thấp. Nếu giảng viên yêu cầu quy mô lớn theo dung lượng, độ phân giải hoặc hàng triệu ảnh, lựa chọn hiện tại cần được thay đổi hoặc bổ sung theo tiêu chí mới.

Khả năng bổ sung các dataset ban đầu

Diabetes và nhà ở thuộc dữ liệu dạng bảng: các cột không tự có quan hệ lân cận như pixel. Nếu dùng CNN 1D, cần giải thích cách sắp thứ tự đặc trưng và so với baseline phù hợp. Đánh giá quần áo là văn bản: cần xử lý thiếu dữ liệu, tokenize, embedding, chọn mục tiêu và tránh rò rỉ nhãn. Cả ba hướng đều là bài toán riêng chứ không chỉ đổi đường dẫn file.

Trong bài nộp này, các hạn chế được công khai và phạm vi được giữ nhất quán. Tập trung vào ba dataset ảnh giúp dành thời gian kiểm chứng gradient, thống nhất ba backend, lặp seed và phân tích vai trò thành phần CNN, thay vì mở rộng số dataset nhưng thiếu chiều sâu giải thích.

CHƯƠNG 4. THIẾT KẾ CNN

4.1. CNN cơ bản làm đối chứng

Baseline có hai khối convolution và pooling, sau đó Flatten, Dense 64 và lớp logits. Mục tiêu là một cấu trúc đủ nhỏ để cài đặt NumPy rõ ràng nhưng vẫn có khả năng học đặc trưng ảnh. Baseline là đối chứng cho improved trong cùng pipeline, không phải mô hình mạnh nhất có thể thiết kế cho CIFAR.

Thứ tự	Khối	Chức năng
1	Conv 3×3: Cin → 8, padding 1	Học 8 đáp ứng cục bộ ban đầu
2	ReLU → MaxPool 2×2	Phi tuyến và giảm không gian lần 1
3	Conv 3×3: 8 → 16, padding 1	Kết hợp đặc trưng vào 16 kênh
4	ReLU → MaxPool 2×2	Phi tuyến và giảm không gian lần 2
5	Flatten → Dense 64 → ReLU	Tổng hợp đặc trưng theo vị trí
6	Dense 64 → C	Trả về logits cho C lớp

Bảng 23. Kiến trúc baseline theo thứ tự forward.

Hai pooling đưa MNIST từ 28×28 về 7×7, CIFAR từ 32×32 về 8×8. Phần Dense nhận độ dài khác nhau nhưng đều có 64 đơn vị ẩn. Lớp cuối chỉ thay số đầu ra C: 10 cho MNIST/CIFAR-10 và 100 cho CIFAR-100. Các siêu tham số còn lại được dùng thống nhất.

```
# Pseudocode of the baseline forward path
x = pool(relu(conv1(x)))
x = pool(relu(conv2(x)))
x = flatten(x)
x = relu(dense1(x))
logits = dense2(x)
```

Mã 7. Mã giả mô tả luồng baseline; các lớp thực nằm trong notebook.

Pseudocode làm rõ thứ tự, nhưng chưa thể hiện trạng thái cache hay hướng gradient. Các chi tiết đó được triển khai trong chương NumPy. Việc tách mô tả kiến trúc khỏi vòng lặp huấn luyện giúp cùng cấu trúc được biểu diễn bằng ba hệ công cụ mà không trộn lẫn dữ liệu với model.

4.2. CNN cải tiến và vị trí can thiệp

Vị trí	Baseline	Improved
Sau Conv1	ReLU	BN → ReLU
Sau Conv2	ReLU	BN → ReLU
Trước Pool2	Đi thẳng tới Pool2	$\text{ReLU}(\text{BN}(\text{Conv}(x)) + x)$
Sau Dense 64	ReLU	BN → ReLU → Dropout 0,25
Lớp cuối	Dense C	Dense C

Bảng 24. Các thay đổi được kết hợp trong improved.

BN xuất hiện bốn lần: sau Conv1, sau Conv2, trong nhánh residual và sau Dense 64. Residual được đặt ở 16 kênh trước pooling lần hai để hai nhánh có cùng shape. Dropout chỉ tác động vector phân loại, tránh phải bổ sung một định nghĩa dropout không gian khác cho từng backend.

$$x_2 = \text{ReLU}(\text{BN}_2(\text{Conv}_2(x_1))), \quad h = \text{ReLU}(\text{BN}_r(\text{Conv}_r(x_2)) + x_2)$$

Bản thử ban đầu chưa có BN sau Dense và gặp train/validation loss cao trên CIFAR-100. Phiên bản cuối bổ sung BN ở vị trí này rồi áp dụng đồng nhất cho cả ba backend. Bốn lượt pilot được giữ riêng; chương kết quả trình bày cả dữ liệu thử để giải thích quá trình điều chỉnh.

Kỳ vọng và điều cần kiểm tra

BN có thể hỗ trợ tối ưu biểu diễn ở các tầng trung gian. Đường cộng có thể giúp nhánh convolution học phân biến đổi bổ sung. Dropout có thể giảm phụ thuộc vào một số đặc trưng. Tuy nhiên, ba cơ chế cùng thay đổi tạo tương tác; chênh lệch giữa baseline và improved không cho biết riêng thành phần nào tạo toàn bộ lợi ích.

Vì vậy nhóm ablation bắt đầu từ improved đầy đủ rồi tắt từng thành phần. Các biến thể không được giới thiệu như những “cải tiến hơn nữa”; chúng là công cụ để kiểm tra giả thuyết. Mọi kết quả, kể cả khi một thành phần không giúp hoặc làm chất lượng giảm, đều phải được giữ trong bảng.

4.3. Theo dõi shape và đếm tham số

Vị trí	MNIST	CIFAR-10/100
Đầu vào	$N \times 1 \times 28 \times 28$	$N \times 3 \times 32 \times 32$
Sau Conv1	$N \times 8 \times 28 \times 28$	$N \times 8 \times 32 \times 32$
Sau Pool1	$N \times 8 \times 14 \times 14$	$N \times 8 \times 16 \times 16$
Sau Conv2 / residual	$N \times 16 \times 14 \times 14$	$N \times 16 \times 16 \times 16$
Sau Pool2	$N \times 16 \times 7 \times 7$	$N \times 16 \times 8 \times 8$
Sau Flatten	$N \times 784$	$N \times 1.024$
Sau Dense 64	$N \times 64$	$N \times 64$
Logits	$N \times 10$	$N \times 10$ hoặc $N \times 100$

Bảng 25. Shape NCHW xuyên suốt hai kiến trúc.

Dataset	Baseline	Improved	Chênh lệch
MNIST	52.138	54.666	2.528
CIFAR-10	67.642	70.170	2.528
CIFAR-100	73.492	76.020	2.528

Bảng 26. Tổng tham số trainable được kiểm tra trong chương trình.

Convolution bổ sung trong residual có $16 \times 16 \times 3 \times 3 + 16 = 2.320$ tham số. Bốn BN có tổng $2 \times (8 + 16 + 16 + 64) = 208$ tham số trainable. Cộng hai phần được 2.528, đúng mức tăng của improved trên cả ba dataset. Dropout, phép cộng và các activation không làm tăng số tham số.

Biến thể no_bn bỏ 208 tham số nhưng giữ nguyên convolution và Dense. Biến thể no_skip và no_dropout giữ nguyên số tham số của improved. Khi so sánh chất lượng, cần đọc cả thông tin này: không nên quy mọi chênh lệch cho chỉ một khái niệm “mạng sâu hơn” hoặc “mạng nhiều tham số hơn”.

4.4. Lý do lựa chọn siêu tham số

Lựa chọn	Lý do trong bài	Đánh đổi / chưa kiểm chứng
Kernel 3×3	Cửa sổ nhỏ, dễ mô tả; giữ shape bằng padding 1	Chưa khảo sát 1×1, 5×5 hoặc dilation
8 và 16 kênh	Giảm chi phí của bản NumPy và kiểm tra gradient	Khả năng biểu diễn CIFAR còn hạn chế
Dense 64	Bộ phân loại gọn và cùng độ rộng ở mọi dataset	Flatten khiến Dense vẫn chiếm đa số tham số
Batch 128	Minibatch vừa phải trong bộ nhớ máy	BN và tốc độ có thể đổi với batch khác
Adam 0,001	Cấu hình cố định, thuận tiện đối chiếu ba backend	Chưa tìm learning rate tốt nhất
5 / 10 / 12 epoch	Ngân sách phù hợp thực hành từ đầu	Chưa chứng minh mọi lượt đã hội tụ
Dropout 0,25	Mức che vừa phải ở phần phân loại	Chưa khảo sát dải xác suất dropout

Bảng 27. Các quyết định thiết kế và giới hạn tương ứng.

Một lựa chọn hợp lý về kỹ thuật không tự trở thành lựa chọn tối ưu về thực nghiệm. Bài không có tìm kiếm lưới rộng để xác định kernel, số kênh, learning rate hoặc dropout tốt nhất. Các thông số được giữ ổn định để tập trung vào tính tương ứng giữa backend, ảnh hưởng của seed và các can thiệp kiến trúc.

Ngân sách epoch giống nhau trong một dataset giúp đối chiếu điều kiện huấn luyện. Nhưng model có số phép tính khác nhau, nên cùng epoch không đồng nghĩa cùng số giây hoặc cùng năng lượng. Hai cách đo công bằng này trả lời hai câu hỏi khác nhau; báo cáo chọn công bằng theo dữ liệu và số lượt cập nhật, đồng thời công khai thời gian thực tế.

Nếu muốn tối ưu accuracy cho ứng dụng, cần một giai đoạn khác: tìm siêu tham số bằng train/validation, quy định ngân sách rõ ràng, rồi đánh giá trên một holdout mới. Giai đoạn đó không được trộn vào bảng hiện tại để tránh thay đổi điều kiện sau khi đã thấy test.

4.5. Bảo đảm ba backend tương ứng

Yếu tố	NumPy	PyTorch	TensorFlow
Ảnh trong model	NCHW	NCHW	NHWC
Conv weights	OIHW	OIHW	HWIO
Dense weights	$D_{in} \times D_{out}$	$D_{out} \times D_{in}$	$D_{in} \times D_{out}$
Trước Flatten	NCHW	NCHW	Permute về NCHW
Phương sai BN	Tổng thể	Lớp tùy chỉnh	fused=False
Khởi tạo	default_rng(seed)	Nạp từ NumPy	Nạp từ NumPy
Gradient	Tự viết	Autograd	GradientTape

Bảng 28. Những khác biệt biểu diễn cần được chuyển đổi đúng.

Tạo model của từng framework với cùng seed là chưa đủ để bảo đảm cùng trọng số, vì bộ sinh số ngẫu nhiên và trình tự khởi tạo có thể khác. Bài tạo trạng thái ban đầu bằng NumPy một lần cho mỗi cấu hình/seed, sau đó sao chép theo đúng layout. Cách kiểm soát trực tiếp này được xác nhận bằng logits và gradient trên batch chung.

Các đối chiếu số học tắt dropout để loại nguồn ngẫu nhiên ở forward. Chúng dùng CPU, tensor nhỏ và dung sai số thực, đồng thời kiểm tra cả train và eval sau cập nhật thống kê BN. Huấn luyện chính vẫn bật dropout ở improved và dùng thiết bị có sẵn theo backend.

Dù cùng trọng số và thứ tự batch, đường học không cần trùng hoàn toàn. Bộ sinh mask dropout riêng của mỗi backend, thứ tự phép cộng trên GPU và cách đặt epsilon trong Adam có thể tạo chênh lệch. Kết quả nhiều seed giúp đọc mức biến thiên, nhưng không phải phép chứng minh rằng một thư viện “tính sai” khi accuracy khác một chút.

So sánh framework trong bài chủ yếu kiểm tra khả năng biểu diễn cùng mô hình và hoàn thành quy trình học. Xếp hạng tốc độ yêu cầu một thiết kế benchmark riêng với cùng phần cứng, mức tối ưu, warm-up, lịch chạy và giới hạn tài nguyên.

4.6. Giao thức nhiều seed và ablation

Nhóm	Tổ hợp	Số lượt
Thực nghiệm chính	3 dataset $\times$ 3 backend $\times$ 2 model $\times$ 3 seed	54
Ablation	2 CIFAR $\times$ 3 can thiệp $\times$ 3 seed; PyTorch	18
Tổng trong giao thức	54 + 18; có 18 lượt seed 42 được tái sử dụng	72
Pilot ngoài giao thức	Các thử nghiệm phiên bản trước	4

Bảng 29. Số lượt và phạm vi của bản mở rộng.

Tên biến thể	Can thiệp	Điều giữ nguyên
no_bn	Thay cả 4 BN bằng identity	Conv, Dense, skip, dropout
no_skip	Bỏ +x trong residual	Conv và BN của nhánh F, toàn bộ tham số
no_dropout	Đặt xác suất dropout bằng 0	Kiến trúc còn lại và số tham số

Bảng 30. Định nghĩa chính xác của ba phép can thiệp.

Mỗi ablation được khởi tạo từ cùng bộ trọng số improved của seed tương ứng rồi mới áp dụng can thiệp. Do đó các trọng số Conv/Dense còn tồn tại giống nhau ở thời điểm đầu. Mỗi lượt vẫn được huấn luyện độc lập; không lấy checkpoint đã học của full để tiếp tục như một mô hình ablation.

Chênh lệch ablation được định nghĩa bằng accuracy của biến thể trừ accuracy full, cùng dataset và seed. Dấu âm nghĩa là tắt thành phần làm kém hơn trong điều kiện này; dấu dương nghĩa là biến thể đạt tốt hơn. Bài tính trung bình và SD của chính các chênh lệch ghép cặp, không suy ra SD chênh lệch chỉ từ hai SD biên.

Giao thức được lưu trước khi tổng hợp kết quả mở rộng. Các seed không được lọc để chỉ giữ lượt tốt. Kiểm tra ablation xác nhận số lớp bị tắt, số tham số, trọng số chung, đầu ra và gradient trước các lượt chạy đầy đủ. Chỉ ba can thiệp riêng lẻ được xét; chưa khảo sát toàn bộ tương tác giữa các thành phần.

CHƯƠNG 5. CNN TỪ ĐẦU BẰNG NUMPY

5.1. Giao diện lớp và tổ chức chương trình

Module `numpy_cnn.py` không sử dụng `autograd`, `PyTorch` hoặc `TensorFlow`. Mỗi lớp tự tính `forward`, `backward` và cung cấp tham số. Phần CNN chỉ tổ chức chuỗi lớp; `optimizer` chỉ đọc các cặp trọng số/gradient. Thiết kế này tách ba nhiệm vụ để có thể kiểm tra từng phần mà không phải chạy một lượt huấn luyện dài.

```
# Common interface (simplified)
class Layer:
    def params(self):
        return []
    def state(self):
        return {}

def network_forward(layers, x, training=True):
    for layer in layers:
        x = layer.forward(x, training)
    return x

def network_backward(layers, gradient):
    for layer in reversed(layers):
        gradient = layer.backward(gradient)
    return gradient
```

Mã 8. Giao diện và chiều duyệt của `forward/backward`, rút gọn từ source.

`params` trả về các cặp (trọng số, gradient) sau `backward`; các lớp không có trọng số trả danh sách rỗng. `state` trả các mảng cần lưu để suy luận, bao gồm cả running statistics của BN. Nhờ phân biệt hai giao diện, trạng thái BN không bị đưa nhầm vào Adam như một tham số có gradient.

Lớp	Tham số học	State bổ sung
Conv2D / Dense	Weight, bias	Không
BatchNorm	Gamma, beta	Running mean và variance
ReLU / Pool / Flatten	Không	Cache tạm khi train
Dropout	Không	Mask tạm và RNG
Residual	Tham số nhánh Conv + BN	State của BN trong nhánh

Bảng 31. Tham số, state và cache của các lớp `NumPy`.

Cache chỉ phục vụ `backward` của `forward` hiện tại; không phải trọng số học và không cần nằm trong checkpoint suy luận. Eval không được cập nhật running statistics hoặc tạo mask dropout mới. Sự khác biệt này được truyền bằng tham số `training` trong mọi lần gọi lớp.

5.2. Convolution forward bằng im2col

Vòng lặp trực tiếp dễ hiểu nhưng tốn chi phí Python khi ảnh và batch lớn. Bản chính dùng `sliding_window_view` để tạo các cửa sổ, đưa mỗi cửa sổ thành một hàng và thực hiện một phép nhân ma trận. Đây là cách tổ chức lại cùng tổng tích, không thay đổi phép toán học của convolution.

```
# Core operations from Conv2D.forward
n, c, h, w = x.shape
xp = np.pad(x, ((0,0), (0,0), (p,p), (p,p)))
win = sliding_window_view(xp, (k,k), axis=(2,3))
oh, ow = win.shape[2:4]
cols = win.transpose(0,2,3,1,4,5).reshape(n*oh*ow, -1)
kernel_matrix = weights.reshape(out_channels, -1)
flat_output = cols @ kernel_matrix.T + bias
y = flat_output.reshape(n,oh,ow,-1).transpose(0,3,1,2)
```

Mã 9. Những phép biến đổi chính của `im2col`; tên biến được viết rõ để giải thích.

Mảng	Shape	Ý nghĩa
x	$N \times C_{in} \times H \times W$	Batch ảnh NCHW
cols	$(N \cdot H_{out} \cdot W_{out}) \times (C_{in} \cdot K^2)$	Mỗi hàng là một cửa sổ
kernel_matrix	$C_{out} \times (C_{in} \cdot K^2)$	Mỗi hàng là một kernel đầu ra
flat_output	$(N \cdot H_{out} \cdot W_{out}) \times C_{out}$	Kết quả tổng tích của mọi cửa sổ

Bảng 32. Shape trong phép nhân ma trận convolution.

Thứ tự transpose trước reshape xác định cách xếp kênh và vị trí kernel vào một hàng. Nếu thứ tự này không trùng thứ tự flatten của trọng số, kết quả sẽ sai dù phép nhân vẫn hợp lệ. Ví dụ số học và đối chiếu với framework kiểm tra chính sự tương ứng này.

`sliding_window_view` ban đầu có thể là một view dùng chung bộ nhớ, nhưng transpose/reshape và phép nhân sau đó có thể tạo mảng trung gian. Lợi ích tốc độ đi kèm chi phí bộ nhớ `im2col`. Bài giữ số kênh nhỏ và batch 128 để phù hợp tài nguyên máy; không tuyên bố đây là cách convolution tiết kiệm bộ nhớ nhất.

5.3. Convolution backward và col2im

$$dW = G^T X_{col}, \quad db = \sum_i G_i$$

$$dX_{col} = GW$$

G ở đây là gradient đầu ra đã đổi thành ma trận một hàng cho mỗi vị trí và mẫu. Hai phép nhân tạo gradient trọng số và gradient các cửa sổ. Bước cuối phải đưa gradient cửa sổ về đúng pixel của ảnh đệm, cộng tất cả đóng góp tại vị trí chồng lấn.

```
# Core operations from Conv2D.backward
g = dy.transpose(0,2,3,1).reshape(-1, out_channels)
dw = (g.T @ cols).reshape(weights.shape)
db = g.sum(axis=0)
dc = (g @ weights.reshape(out_channels, -1))
dc = dc.reshape(n, oh, ow, c, k, k)
dxp = np.zeros((n,c,h+2*p,w+2*p), dtype=np.float32)
for u in range(k):
    for v in range(k):
        part = dc[:, :, :, :, u, v].transpose(0,3,1,2)
        dxp[:, :, u:u+oh, v:v+ow] += part
dx = dxp[:, :, p:p+h, p:p+w]
```

Mã 10. Cộng dồn col2im bằng += và cắt phần padding.

Phép += là điểm quan trọng: cùng một pixel nhận gradient từ nhiều cửa sổ. Dùng phép gán = sẽ làm mất đóng góp trước đó. Các vòng lặp chỉ chạy theo hai chiều kernel; việc cộng cho tất cả ảnh và kênh vẫn được vector hóa bằng NumPy.

Gradient kernel trong ví dụ	Giá trị
Hàng 1	1,598365, 0,519138
Hàng 2	0,332096, 0,643372

Bảng 33. Gradient kernel 2×2 từ ví dụ xuyên suốt.

Gradient trọng số đã bao gồm yếu tố trung bình theo batch thông qua gradient loss đi xuống, nên không chia N thêm ở convolution. Kiểm tra lớp Conv dùng loss tuyến tính nhỏ với gradient đầu ra cho trước để đối chiếu dW, db và dX; cách này cô lập convolution khỏi các phần còn lại của CNN.

5.4. ReLU, MaxPool và Flatten

Các lớp không có tham số vẫn cần backward chính xác. ReLU lưu mask; MaxPool lưu argmax; Flatten lưu shape đầu vào. Nếu bỏ các lớp này trong chuỗi backward, gradient từ Dense không thể trở lại đúng tensor convolution.

```
# ReLU: same mask is reused for the current backward
mask = x > 0
y = np.maximum(x, 0)
dx = dy * mask

# Flatten: no learned parameter
original_shape = x.shape
flat = x.reshape(len(x), -1)
restored_gradient = dflat.reshape(original_shape)
```

Mã 11. Mask và shape là đủ cho hai lớp đơn giản.

MaxPool gom mỗi cửa sổ 2×2 thành bốn phần tử, lưu chỉ số cực đại ở trục cuối. Backward tạo mảng 0 có cùng bố cục cửa sổ, đặt gradient vào chỉ số đã lưu bằng `put_along_axis`, rồi đảo reshape/transpose để thu ảnh gradient.

```
# Scatter the upstream gradient to the recorded maxima
routed = np.zeros((*dy.shape, 4), dtype=np.float32)
np.put_along_axis(routed, argmax[...], dy[...], axis=-1)
dx = routed.reshape(n,c,oh,ow,2,2)
dx = dx.transpose(0,1,2,4,3,5).reshape(n,c,oh*2,ow*2)
```

Mã 12. Bước đưa gradient pooling về vị trí cực đại.

Tình huống	Quy ước trong source
Hai giá trị cực đại bằng nhau	argmax chọn vị trí đầu theo thứ tự flatten cửa sổ
Chiều không chia hết cho 2	Phần cuối không đủ cửa sổ bị bỏ ở forward
Gradient phần bị bỏ	Bằng 0 khi tạo lại tensor đầu vào
Shape thực nghiệm	28 và 32 đều chia hết qua hai pooling

Bảng 34. Các trường hợp biên của pooling.

Kiểm tra gradient số chọn đầu vào tránh cực đại bằng nhau để không vướng điểm không trơn. Tuy nhiên source vẫn phải có quy ước xác định cho trường hợp hòa, vì trong ảnh thật các vùng bằng 0 sau ReLU có thể tạo nhiều phần tử bằng nhau.

5.5. Dense và loss ổn định số học

```
# Dense forward and backward
logits = features @ weights + bias
dweights = features.T @ dlogits
dbias = dlogits.sum(axis=0)
dfeatures = dlogits @ weights.T

# Stable sparse softmax cross-entropy
z = logits - logits.max(axis=1, keepdims=True)
logsum = np.log(np.exp(z).sum(axis=1, keepdims=True))
loss = (logsum[:,0] - z[np.arange(len(labels)), labels]).mean()
gradient = np.exp(z - logsum)
gradient[np.arange(len(labels)), labels] -= 1
gradient /= len(labels)
```

Mã 13. Công thức Dense và cross-entropy dùng trực tiếp trong NumPy.

Loss được tính bằng log-sum-exp thay vì lấy log của một softmax có thể bị làm tròn về 0. Dù logits rất lớn, việc trừ cực đại giúp exp lớn nhất bằng 1. Các xác suất còn lại có thể rất nhỏ nhưng biểu thức loss vẫn tránh bước $\log(0)$ không cần thiết.

Hàng Dense của ví dụ	Gradient theo ba đầu ra
1	3,11448, -4,15120, 1,03672
2	3,73738, -4,98145, 1,24407
3	0,62290, -0,83024, 0,20734
4	1,24579, -1,66048, 0,41469

Bảng 35. Gradient ma trận Dense 4×3 trong ví dụ nhân tạo.

Mỗi hàng gradient Dense bằng một đặc trưng đầu vào nhân với vector gradient logits. Vì ví dụ chỉ có một mẫu, phép lấy trung bình theo batch không làm thay đổi thang. Khi có nhiều mẫu, phép nhân $X^T G$ cộng đóng góp đúng theo hàng tương ứng.

Các kiểm tra có ý nghĩa

Kiểm tra ổn định loss đưa logits lớn vào hàm và yêu cầu loss/gradient hữu hạn. Kiểm tra gradient Dense so đạo hàm số của trọng số, bias và đầu vào. Kiểm tra khả năng học bài toán nhỏ xác nhận optimizer phối hợp được với mạng. Ba loại kiểm tra bắt những lỗi khác nhau và không nên thay thế lẫn nhau.

5.6. BatchNorm forward và backward

BN lưu hai loại trạng thái: z đã chuẩn hóa và nghịch đảo độ lệch chuẩn cho backward hiện tại; running mean/variance cho eval sau này. Ở tensor NCHW, mọi phép giảm chiều đều cần dùng đúng $\text{axes}=(0,2,3)$, giữ riêng thông tin từng kênh.

$$d\gamma = \sum_i g_i \hat{x}_i, \quad d\beta = \sum_i g_i$$

$$dx_i = \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}} (g_i - \bar{g} - \hat{x}_i \overline{g\hat{x}})$$

```
axes = (0, 2, 3)
dgamma = (g * normalized).sum(axis=axes)
dbeta = g.sum(axis=axes)
centered = g - g.mean(axis=axes, keepdims=True)
centered -= normalized * (g * normalized).mean(axis=axes, keepdims=True)
dx = centered * (gamma * inv_std)[None, :, None, None]
```

Mã 14. Đạo hàm BN dùng trung bình theo đúng nhóm chuẩn hóa.

Công thức dX rút gọn chứa ba hạng: gradient trực tiếp, hiệu chỉnh do mean và hiệu chỉnh do variance. Nếu chỉ nhân gradient với $1/\text{std}$ như khi mean/variance là hằng số, đạo hàm train sẽ sai vì thống kê batch cũng phụ thuộc vào x . Ở eval, hành vi khác do running statistics được coi là trạng thái cố định.

Tái sử dụng cho Dense

BatchNorm1D chuyển $N \times D$ thành $N \times D \times 1 \times 1$ rồi gọi cùng lớp BatchNorm2D. $\text{Axes}=(0,2,3)$ khi đó tương đương chỉ lấy thống kê trên N . Sau forward hoặc backward, hai chiều độ dài 1 được bỏ. Cách dùng chung này giảm lặp công thức và cho phép kiểm tra riêng cả dạng ảnh lẫn dạng vector.

Running variance trong bài dùng phương sai tổng thể, thống nhất với NumPy và lựa chọn Keras. Đây là chi tiết cần ghi rõ khi chuyển checkpoint giữa framework. Sai khác running statistics có thể không xuất hiện ở logits train đầu tiên nhưng lộ ra khi đổi sang eval, vì vậy kiểm tra cần bao gồm cả hai chế độ.

5.7. Residual và dropout thủ công

```
# Residual forward
branch = conv.forward(x, training)
branch = bn.forward(branch, training)
y = relu.forward(branch + x, training)

# Residual backward
g = relu.backward(dy)
dx = g + conv.backward(bn.backward(g))
```

Mã 15. Gradient residual cộng hai nhánh sau mask ReLU.

Phải áp dụng backward của ReLU ngoài cùng trước khi tách gradient về hai nhánh. Cả nhánh identity và nhánh F nhận cùng g đã đi qua mask. Việc cộng trực tiếp dy với gradient nhánh F sẽ sai ở những vị trí tổng trước ReLU không dương.

```
# Inverted dropout, p = 0.25
if training:
    mask = (rng.random(x.shape) >= p).astype(np.float32)
    mask /= 1 - p
    y = x * mask
else:
    y = x
# Backward uses the mask from the same forward:
dx = dy * mask
```

Mã 16. Dropout dùng một mask cho forward và backward của cùng batch.

Dòng backward chỉ áp dụng sau forward ở train; eval không gọi backward và không cần mask. Tạo lại một mask ngẫu nhiên trong backward sẽ làm đạo hàm không còn thuộc forward đã tính. Trong source, RNG được truyền từ model và dùng tiếp qua các batch.

Can thiệp	Thay đổi phép tính	Tham số
Tắt dropout	$y=x$ ở cả train và eval	Giữ nguyên
Tắt skip	$y=\text{ReLU}(F(x))$	Giữ Conv và BN
Tắt BN	Thay chuẩn hóa bằng identity	Bỏ gamma/beta BN

Bảng 36. Liên hệ thuật toán thủ công với định nghĩa ablation.

Các ablation được triển khai bằng PyTorch để chạy bổ sung trên GPU. Bản NumPy trong nhóm chính vẫn dùng đúng baseline và improved đầy đủ. Việc giải thích ablation ở đây giúp người đọc nổi can thiệp framework với công thức forward/backward đã học.

5.8. Cài đặt Adam và trạng thái optimizer

```
# One update for a parameter array and its gradient
m *= beta1
m += (1 - beta1) * gradient
v *= beta2
v += (1 - beta2) * gradient * gradient
m_hat = m / (1 - beta1 ** step)
v_hat = v / (1 - beta2 ** step)
weights -= learning_rate * m_hat / (np.sqrt(v_hat) + epsilon)
```

Mã 17. Adam cập nhật tại chỗ sau khi tăng bộ đếm step.

Mỗi mảng trọng số có một cặp trạng thái m và v cùng shape. Chúng bắt đầu bằng 0 ở bước đầu tiên. Bộ đếm step tăng một lần cho mỗi minibatch, không tăng riêng cho từng mảng trong model. Nếu tăng step trong vòng lặp tham số, các lớp sẽ dùng hiệu chỉnh bias khác nhau dù thuộc cùng một bước học.

Kernel ví dụ	Trước cập nhật	Sau một bước Adam
Hàng 1	1,0000, 0,0000	0,9990, -0,0010
Hàng 2	0,0000, -1,0000	-0,0010, -1,0010

Bảng 37. Bước Adam đầu tiên với learning rate 0,001 trong ví dụ.

Ở bước đầu, khi gradient không bằng 0 và bỏ qua tác động rất nhỏ của epsilon, hiệu chỉnh bias làm tỷ số gần với dấu của gradient. Điều đó giải thích vì sao các phần tử kernel trong ví dụ thay đổi gần 0,001 dù độ lớn gradient khác nhau. Ở các bước sau, lịch sử m/v làm cập nhật không còn đơn giản như vậy.

Checkpoint suy luận và tiếp tục huấn luyện

Checkpoint của bài lưu trọng số và running statistics để khôi phục dự đoán. Nó không lưu đầy đủ m , v , step và trạng thái RNG nhằm tiếp tục chính xác một lượt đang dở. Vì vậy khi một lượt chưa có metrics hoàn chỉnh bị ngắt, runner bắt đầu lại lượt đó từ đầu; không tuyên bố đã resume đúng quỹ đạo tối ưu.

Nếu mở rộng chức năng resume, cần lưu optimizer state, epoch, vị trí batch và RNG cho mọi thư viện liên quan. Chỉ tải weights rồi tạo Adam mới là một lượt huấn luyện tiếp với trạng thái tối ưu khác, dù model có cùng dự đoán ngay khi vừa tải.

5.9. Vòng lặp huấn luyện và chọn checkpoint

```
# Simplified control flow; logging and batching are in src/train.py
best_val_loss = float('inf')
for epoch in range(1, epochs + 1):
    for images, labels in training_batches(epoch):
        logits = model.forward(images, training=True)
        loss, gradient = cross_entropy(logits, labels)
        model.backward(gradient)
        optimizer.step(model.params())
    val_logits = evaluate_validation(model)
    val_loss, _ = cross_entropy(val_logits, validation_labels)
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        save_checkpoint(model)
    restore_best_checkpoint(model)
test_logits = evaluate_test(model)
```

Mã 18. Mã giả nhấn mạnh vị trí validation và test trong vòng lặp.

Train loss được tích lũy từ các batch trong khi trọng số đang thay đổi. Validation được tính sau epoch với một trạng thái model cố định, dropout tắt và BN dùng running statistics. Hai đường vì thế không hoàn toàn cùng điều kiện đo; validation tốt hơn train ở một epoch không tự động là dấu hiệu dữ liệu bị rò rỉ.

Mỗi epoch ghi train loss/accuracy, validation loss/accuracy và thời gian. Nếu loss không hữu hạn, chương trình dừng thay vì ghi một kết quả như hợp lệ. Cuối lượt, checkpoint tốt nhất được khôi phục, chạy toàn bộ test và lưu logits cùng nhãn để có thể tính lại mọi chỉ số chính.

File	Thông tin chính
history.csv	Một dòng cho mỗi epoch
config.json	Seed, split, phiên bản, thiết bị và siêu tham số
metrics.json	Chỉ số test, best_epoch, số tham số và thời gian
test_outputs.npz	Logits và nhãn test chưa làm tròn
weights.npz	Trạng thái model NumPy được chọn

Bảng 38. Các đầu ra nổi quá trình học với báo cáo.

5.10. Kiểm chứng bản từ đầu

Bộ kiểm tra gốc có chín test bao phủ gradient Conv, Dense, BN ảnh, BN vector, pooling, residual; ổn định loss; shape/số tham số và khả năng học một bài toán nhỏ. Các tensor nhỏ giúp kiểm tra nhanh, đồng thời có thể cô lập nơi sai nếu một phép đổi chiều không đạt.

$$g_{num,i} = \frac{L(\theta + h e_i) - L(\theta - h e_i)}{2h}$$

```
old = weights[index]
weights[index] = old + h
plus = scalar_objective()
weights[index] = old - h
minus = scalar_objective()
weights[index] = old
numerical_gradient = (plus - minus) / (2 * h)
np.testing.assert_allclose(analytic_gradient, numerical_gradient,
                           atol=atol, rtol=rtol)
```

Mã 19. Khôi phục trọng số sau hai phép nhiễu khi tính gradient số.

Đổi chiều sử dụng các phần tử được chọn, không duyệt mọi phần tử của mọi tensor lớn. Bước sai phân trong bộ kiểm tra là 0,002 với dung sai phù hợp float32. ReLU và max pooling có điểm không trơn, nên đầu vào kiểm tra được chọn để hạn chế tình huống hòa hoặc đúng tại ngưỡng.

Loại bằng chứng	Có thể kết luận	Không đủ để kết luận
Gradient số	Đạo hàm khớp ở trường hợp được thử	Mọi đầu vào đều đã được chứng minh
Học bài toán nhỏ	Forward, loss và optimizer phối hợp được	Model tổng quát tốt trên CIFAR
Logits/gradient khớp framework	Các cài đặt tương ứng ở batch thử	Huấn luyện mọi seed sẽ giống từng bit
Test thật	Chất lượng trên split đánh giá đã nêu	Hiệu quả ở mọi nguồn ảnh mới

Bảng 39. Phạm vi diễn giải của từng kiểm tra.

Ví dụ số học xuyên suốt được thực thi độc lập và đổi chiều convolution bằng vòng lặp trực tiếp. Kết quả của các kiểm tra được đưa vào notebook và file verification, giúp người đọc kiểm tra được một khẳng định kỹ thuật thay vì chỉ tin vào mô tả bằng lời.

CHƯƠNG 6. CÀI ĐẶT BẰNG PYTORCH

6.1. Module và đồ thị tự động tính gradient

TorchCNN kế thừa `nn.Module` và sử dụng `nn.Sequential` cho chuỗi lớp. Convolution, Dense, ReLU, pooling và dropout được biểu diễn bằng các module tương ứng. PyTorch ghi lại quan hệ giữa các phép toán tensor khi gradient được bật; backward sử dụng đồ thị đó để tính đạo hàm [17]. Người viết vẫn phải định nghĩa đúng forward và vòng lặp học.

```
# Baseline structure; the complete class also supports improved
layers = [
    nn.Conv2d(channels, 8, 3, padding=1),
    nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(8, 16, 3, padding=1),
    nn.ReLU(), nn.MaxPool2d(2), nn.Flatten(),
    nn.Linear(16 * (size // 4) ** 2, 64),
    nn.ReLU(), nn.Linear(64, classes)
]
network = nn.Sequential(*layers)
logits = network(images)
```

Mã 20. Cấu trúc baseline tương ứng bản NumPy.

Một forward đúng shape chưa đủ: thứ tự BN, ReLU, pooling và residual phải trùng bản thiết kế. Improved chèn BN sau hai Conv, một residual block trước Pool2, BN sau Dense 64 và dropout trước lớp cuối. Source chỉ thay đổi những vị trí đã mô tả ở chương 4.

Nhiệm vụ	NumPy	PyTorch
Định nghĩa phép biến đổi	Lớp tự viết	Module và phép toán tensor
Gradient tham số	Tự lập công thức	Autograd đi qua forward
Danh sách tham số	<code>params()</code> của từng lớp	<code>model.parameters()</code>
Lưu trạng thái	Dictionary mảng NumPy	<code>state_dict</code> gồm parameter và buffer

Bảng 40. Ánh xạ trách nhiệm từ NumPy sang PyTorch.

Autograd không phát hiện mọi sai sót về ý nghĩa mô hình. Nếu cộng nhầm tensor, dùng sai nhân hoặc chuẩn hóa hai lần, thư viện vẫn có thể tính đúng đạo hàm của chương trình sai đó. Vì vậy kiểm tra số học và kiểm tra dữ liệu vẫn cần thiết ngay khi dùng framework.

6.2. Khởi tạo chung và BatchNorm tùy chỉnh

```
# Transfer the same initial Dense weights from NumPy
with torch.no_grad():
    torch_layer.weight.copy_(torch.from_numpy(numpy_weight.T.copy()))
    torch_layer.bias.copy_(torch.from_numpy(numpy_bias))

# Population statistics used by the custom image BatchNorm
mean = x.mean((0, 2, 3))
variance = x.var((0, 2, 3), unbiased=False)
```

Mã 21. Chuyển vị Dense và tính phương sai tổng thể theo kênh.

Convolution NumPy và PyTorch đều dùng thứ tự Cout,Cin,K,K, nên không cần hoán vị trục trọng số. Dense NumPy lưu Din,Dout nhưng nn.Linear lưu Dout,Din, nên cần chuyển vị. Việc copy được đặt trong no_grad vì đây là thiết lập tham số ban đầu, không phải một phần của hàm mất mát cần đạo hàm.

PopulationBatchNorm có gamma và beta dưới dạng Parameter; running mean và running variance được đăng ký bằng register_buffer. Buffer được lưu trong state_dict và chuyển thiết bị cùng model, nhưng không nằm trong danh sách tham số Adam cập nhật. Dạng 1D tái sử dụng lớp ảnh bằng hai chiều không gian bằng 1.

```
# Update running statistics without making a gradient graph
with torch.no_grad():
    running_mean.mul_(0.9).add_(mean.detach(), alpha=0.1)
    running_var.mul_(0.9).add_(variance.detach(), alpha=0.1)
normalized = (x - mean[None,:,None,None])
normalized *= torch.rsqrt(variance[None,:,None,None] + 1e-5)
```

Mã 22. Các phép tính BN được viết bằng tensor để autograd xử lý.

Bài dùng BN tùy chỉnh vì muốn running variance khớp định nghĩa của NumPy và Keras đã chọn. Đây là quyết định phục vụ đối chiếu, không phải chỉ dẫn rằng luôn cần thay BatchNorm chuẩn của PyTorch. Khi dùng layer chuẩn trong dự án khác, cần đọc quy ước variance và momentum để hiểu state được cập nhật ra sao.

Sau khi nạp khởi tạo, số tham số được đối chiếu với NumPy bằng assert. Kiểm tra logits train và eval bổ sung bằng chứng mạnh hơn đếm tham số: hai model có cùng số tham số vẫn có thể khác thứ tự hoặc công thức forward.

6.3. Một bước huấn luyện PyTorch

```
model.train()
optimizer.zero_grad(set_to_none=True)
images = torch.from_numpy(np.ascontiguousarray(batch)).to(device)
targets = torch.from_numpy(labels).to(device)
logits = model(images)
loss = torch.nn.functional.cross_entropy(logits, targets)
loss.backward()
optimizer.step()
correct = (logits.argmax(dim=1) == targets).sum().item()
```

Mã 23. Thứ tự thao tác cho một minibatch trong `src/train.py`.

`model.train` bắt hành vi huấn luyện của BN và dropout. `zero_grad` xóa gradient từ bước trước; PyTorch tích lũy gradient vào thuộc tính `grad` nên nếu bỏ bước này, optimizer sẽ dùng tổng gradient của nhiều batch ngoài dự định. `set_to_none=True` đặt gradient về None để thư viện tạo lại khi `backward`.

`cross_entropy` nhận logits và nhãn nguyên, nên không đặt softmax vào model trước hàm này. Thêm softmax không cần thiết có thể khiến loss đang nhận xác suất như thể đó là logits. Trong đánh giá, `argmax` của logits và softmax là cùng nhãn; softmax chỉ cần khi muốn đọc confidence.

Thao tác	Tác dụng	Lỗi thường gặp
<code>to(device)</code>	Đặt model và batch cùng thiết bị	Trộn tensor CPU với CUDA
<code>zero_grad</code>	Bắt đầu gradient của batch mới	Cộng dồn ngoài ý muốn
<code>backward</code>	Tính gradient từ loss	Loss bị detach trước khi gọi
<code>step</code>	Cập nhật tham số theo Adam	Quên cập nhật dù loss đã có gradient

Bảng 41. Các thao tác cần có trong bước học.

Để ghi số liệu, `loss.item()` và số dự đoán đúng được đưa về Python sau forward. Đây là giá trị đã dùng cho batch hiện tại, trước khi model được đánh giá trên validation. Các thời điểm lấy số liệu được giữ thống nhất giữa backend để tránh trộn metric của hai trạng thái model.

Vòng ngoài epoch, quy tắc chọn checkpoint và hàm tạo batch dùng chung với NumPy. PyTorch chỉ cung cấp các hàm `step`, `predict`, `save` và `restore` cho runner; điều này giữ phần điều khiển thí nghiệm nhất quán.

6.4. Eval, lưu và khôi phục checkpoint

```
model.eval()
with torch.no_grad():
    logits = model(images)

torch.save(model.state_dict(), checkpoint_path)
state = torch.load(checkpoint_path, map_location='cpu', weights_only=True)
model.load_state_dict(state)
model.eval()
```

Mã 24. Tách chế độ model, ghi đồ thị và trạng thái checkpoint.

eval và no_grad làm hai việc khác nhau. eval thay hành vi những module như BN và dropout; no_grad tắt việc ghi đồ thị đạo hàm. Chỉ dùng no_grad mà quên eval vẫn có thể tạo mask dropout và dùng thống kê batch. Chỉ dùng eval mà không no_grad vẫn có thể giữ đồ thị không cần thiết khi suy luận.

Checkpoint tốt nhất được lưu bằng state_dict, gồm weight, bias và các buffer BN. Khi tải, chương trình phải tạo đúng kiến trúc rồi nạp trạng thái. Với ablation, cần áp dụng đúng no_bn, no_skip hoặc no_dropout trước khi suy luận; cờ tắt skip và xác suất dropout là cấu hình kiến trúc, không tự được khôi phục chỉ từ các mảng weight.

Thành phần	Lưu ở đâu
Conv/Dense và gamma/beta BN	weights.pt
Running mean/variance	Buffer trong weights.pt
Dataset, seed, ablation	config.json
Epoch được chọn	metrics.json và history.csv
Logits để đối chiếu	test_outputs.npz

Bảng 42. Những dữ liệu cần phối hợp khi khôi phục một lượt.

Bài kiểm tra checkpoint bằng một tiến trình CPU mới: tải model và chạy lại sáu ảnh test cố định, rồi so logits với file đã lưu. Cách kiểm tra này bắt được việc bỏ sót state BN hoặc cấu hình ablation, những lỗi có thể bị che khi model vẫn còn trong bộ nhớ sau huấn luyện.

Sáu ảnh là một phép kiểm tra khôi phục có chủ đích, không thay thế việc đánh giá đủ 10.000 ảnh của mỗi lượt. Metric chính được tính từ toàn bộ test và được tính lại độc lập từ logits. Hai kiểm tra tập trung vào hai rủi ro khác nhau: tính đúng của file kết quả và khả năng sử dụng checkpoint.

6.5. Cài đặt các phép ablation

```
model = TorchCNN(channels, size, classes, variant='improved')
model.load_numpy(shared_initial_state)
model.ablate(component)
```

```
# Within the residual block
branch = self.bn(self.conv(x))
y = torch.relu(branch + x if self.use_skip else branch)
```

Mã 25. Khởi tạo chung trước can thiệp và phép tắt riêng đường cộng.

`no_bn` thay các `PopulationBatchNorm` ở chuỗi chính và trong residual bằng `Identity`. `no_skip` đặt `use_skip=False` cho residual, giữ nguyên `conv` và `BN` của nhánh. `no_dropout` đặt `p=0` ở `Dropout`. `Optimizer` được tạo sau can thiệp để danh sách tham số đúng với model cuối, đặc biệt khi `BN` đã bị loại.

Kiểm tra trước huấn luyện	Điều cần đúng
Trọng số chung	Mọi mảng còn tồn tại khớp improved ban đầu
<code>no_bn</code>	Không còn cả 4 BN; vẫn có 3 convolution
<code>no_skip</code>	Đầu ra và gradient bằng nhánh F qua ReLU
<code>no_dropout</code>	Output bằng input khi train
Gradient sau loss	Mọi tham số còn lại có gradient hữu hạn
Đường dẫn kết quả	Seed và can thiệp khác nhau không ghi đè

Bảng 43. Sáu kiểm tra dành riêng cho thiết kế ablation.

Tắt skip khác với xóa residual block: nếu xóa cả block, số lớp convolution và số tham số cùng giảm, khiến phép so sánh gộp nhiều thay đổi hơn. Bản hiện tại giữ nhánh F để cô lập tốt hơn tác dụng của đường cộng trong kiến trúc đã định. Với `no_bn`, số tham số giảm 208 vì `gamma/beta` biến mất; sự khác biệt này được ghi trong bảng.

Kết quả ablation không cho phép cộng các mức giảm để suy ra toàn bộ lợi ích improved. BN, skip và dropout tương tác với nhau; một thành phần có thể hữu ích khi các phần còn lại hiện diện nhưng không có cùng tác động trong baseline. Khảo sát mọi tổ hợp sẽ là một thiết kế thực nghiệm khác.

6.6. Kiểm chứng PyTorch và quản lý thiết bị

Dataset	Kiến trúc	Sai số logits train	Sai số gradient Conv1
mnist	baseline	1.19e-06	3.50e-07
mnist	improved	5.01e-06	8.64e-06
cifar10	baseline	1.49e-06	6.56e-07
cifar10	improved	4.11e-06	4.92e-06
cifar100	baseline	3.58e-06	5.07e-07
cifar100	improved	4.89e-06	8.11e-06

Bảng 44. Đối chiếu PyTorch với NumPy trên batch kiểm tra CPU.

Mỗi cấu hình còn kiểm tra gradient đầu vào và logits eval sau cập nhật running statistics. Dropout được tắt để loại khác biệt mask ngẫu nhiên. Dung sai dùng atol=2e-4, rtol=3e-4; các sai số quan sát nằm trong giới hạn. Các kiểm tra này không đòi bitwise equality.

```
torch.backends.cudnn.benchmark = False
torch.backends.cudnn.deterministic = True
torch.backends.cuda.matmul.allow_tf32 = False
torch.backends.cudnn.allow_tf32 = False
device = 'cuda' if torch.cuda.is_available() else 'cpu'
model.to(device)
```

Mã 26. Một số thiết lập của lượt huấn luyện PyTorch trong bài.

Những thiết lập này giảm một số nguồn thay đổi thuật toán và độ chính xác phép nhân. Chúng không phải lời bảo đảm mọi GPU, driver và phiên bản thư viện sẽ cho cùng kết quả từng bit. Phiên bản và thiết bị của từng lượt được ghi trong config để người đọc biết điều kiện đã chạy.

PyTorch còn được chọn cho nhóm ablation vì giao diện module giúp thay BN bằng Identity, điều khiển đường cộng và dropout trực tiếp. Bài vẫn giữ hai backend còn lại trong nhóm nhiều seed, nên phân tích thành phần không bị nhầm thành một phép đánh giá ablation đã chạy ở cả ba cách triển khai.

CHƯƠNG 7. CÀI ĐẶT BẰNG TENSORFLOW

7.1. Keras Functional API và nhánh residual

Mô hình TensorFlow dùng Functional API để mô tả rõ luồng tensor và phép cộng nhánh. Input có shape H,W,C không tính batch. Tên các lớp có tham số được đặt cố định như conv1, conv2, res_conv, fc1 để việc nạp trọng số NumPy không phụ thuộc vào tên tự sinh.

```
# Residual part of the improved Keras model
residual = x
x = layers.Conv2D(16, 3, padding='same', name='res_conv')(x)
x = layers.BatchNormalization(momentum=0.9, epsilon=1e-5,
                             fused=False, name='res_bn')(x)
x = layers.Add()([x, residual])
x = layers.ReLU()(x)
```

Mã 27. Nhánh residual giữ nguyên shape để dùng Add.

Hai nhánh được cộng sau Conv và BN, rồi mới qua ReLU. Nếu đổi thành ReLU trong nhánh trước phép cộng mà không có ReLU ngoài, ta đã tạo một kiến trúc khác. Source giữ thứ tự tương ứng với lớp Residual của NumPy và PyTorch để đối chiếu forward/backward.

Khối	Lớp Keras sử dụng
Convolution	Conv2D với padding=same, stride 1
Chuẩn hóa	BatchNormalization, epsilon 1e-5, fused=False
Phi tuyến / giảm không gian	ReLU / MaxPool2D
Cộng residual	Add nhận hai tensor cùng shape
Phân phân loại	Permute → Reshape → Dense → ReLU → Dense
Regularization improved	BN sau Dense 64, Dropout 0,25

Bảng 45. Ánh xạ kiến trúc sang Keras.

Model trả logits ở Dense cuối, không đặt activation softmax. Loss trong vòng lặp nhận logits và nhân nguyên để dùng công thức ổn định của framework. Cách thiết kế tương ứng với bản NumPy, đồng thời tránh việc vô tình áp dụng softmax hai lần.

7.2. Chuyển trọng số và thứ tự Flatten

```
# OIHW (NumPy) -> HWIO (Keras Conv2D)
keras_conv_weight = numpy_conv_weight.transpose(2, 3, 1, 0)
keras_layer.set_weights([keras_conv_weight, numpy_conv_bias])

# Restore the NCHW feature order before Dense
x = layers.Permute((3, 1, 2))(x)
x = layers.Reshape((16 * (size // 4) ** 2,))(x)
```

Mã 28. Chuyển layout của kernel và vector đặc trưng.

Conv2D Keras lưu trọng số theo K,K,Cin,Cout; NumPy lưu Cout,Cin,K,K. Hoán vị (2,3,1,0) đưa đúng bốn trục sang thứ tự Keras. Dense Keras đã dùng Din,Dout nên giữ nguyên ma trận của NumPy. BN nạp gamma, beta, running mean và running variance theo thứ tự mà lớp yêu cầu.

Mảng NumPy	Phép chuyển	Mảng TensorFlow
Ảnh NCHW	transpose(0,2,3,1)	Ảnh NHWC
Kernel OIHW	transpose(2,3,1,0)	Kernel HWIO
Dense Din,Dout	Không chuyển vị	Dense Din,Dout
Feature map NHWC	Permute(3,1,2)	Feature map NCHW trước Flatten

Bảng 46. Bốn phép đổi chiều layout quan trọng.

Permute nhận các trục không tính chiều batch. Sau đó Reshape flatten tensor NCHW thành cùng thứ tự đặc trưng như NumPy/PyTorch. Nếu chỉ flatten NHWC trực tiếp, mọi shape cuối vẫn có thể đúng và số tham số không đổi, nhưng cùng ma trận Dense sẽ nhân với một vector đã hoán vị.

Cách phát hiện lỗi layout

Một phép kiểm tra round-trip tensor bắt được hoán vị sai ở bước chuẩn bị. So logits với trọng số chung bắt được thứ tự Flatten không tương ứng. So gradient convolution đầu tiên còn kiểm tra rằng phép đổi trục ở cả forward và backward đều nhất quán. Đó là lý do chỉ nhìn model.summary chưa đủ.

Khi đọc code chuyển state, cần phân biệt tên lớp Keras với chỉ số lớp trong chuỗi NumPy. Improved có nhiều lớp hơn baseline, nên mapping được định nghĩa riêng cho hai kiến trúc; không dùng một bộ chỉ số rồi giả định mọi vị trí vẫn trùng nhau.

7.3. GradientTape và bước cập nhật

GradientTape ghi các phép toán cần thiết để lấy đạo hàm theo các biến được theo dõi [18]. Trong bài, loss được tính bên trong tape; sau khối đó, chương trình lấy gradient theo trainable_weights rồi đưa các cặp gradient/biến cho Adam. Vòng lặp ngoài vẫn dùng cùng lịch batch và quy tắc validation của hai backend còn lại.

```
@tf.function(reduce_retracing=True)
def train_step(images, labels):
    with tf.GradientTape() as tape:
        logits = model(images, training=True)
        losses = tf.nn.sparse_softmax_cross_entropy_with_logits(
            labels=labels, logits=logits)
        loss = tf.reduce_mean(losses)
    gradients = tape.gradient(loss, model.trainable_weights)
    optimizer.apply_gradients(zip(gradients, model.trainable_weights))
    predicted = tf.argmax(logits, axis=1)
    correct = tf.reduce_sum(tf.cast(predicted == labels, tf.int32))
    return loss, correct
```

Mã 29. Bước huấn luyện tùy chỉnh tương ứng src/train.py.

training=True truyền chế độ đến BN và dropout. Loss của từng mẫu được lấy trung bình một lần; không chia thêm batch sau khi tape.gradient trả về. Nhãn trong pipeline có kiểu int64, phù hợp kiểu mặc định của argmax khi so dự đoán đúng.

Thao tác	Điều cần giữ
Tạo loss	Nằm trong ngữ cảnh GradientTape
Biến cần tối ưu	Danh sách trainable_weights của model
Running statistics	State BN được cập nhật khi training=True
Áp dụng gradient	Ghép đúng thứ tự gradient và biến
Đưa số liệu ra ngoài	Chỉ chuyển về NumPy/Python sau khi bước học hoàn tất

Bảng 47. Các điểm kiểm soát của GradientTape.

Các phép NumPy bên ngoài TensorFlow không tự nằm trong đồ thị gradient. Nếu chuyển một tensor trung gian sang NumPy để tính loss rồi đưa lại vào TensorFlow, liên kết đạo hàm có thể bị mất. Trong bước học, mọi phép từ model đến loss vì thế được giữ bằng toán tử TensorFlow.

7.4. BN, dropout và thực thi graph

Keras BatchNormalization được cấu hình momentum=0,9, epsilon=1e-5 và fused=False. Công thức running là 0,9 lần thống kê cũ cộng 0,1 lần batch mới, trùng cách viết NumPy/PyTorch của bài. Việc dùng tham số momentum mà không đối chiếu công thức dễ tạo nhầm lẫn khi chuyển model.

```
bn = layers.BatchNormalization(momentum=0.9, epsilon=1e-5,
                              fused=False)
dropout = layers.Dropout(0.25)
train_logits = model(images, training=True)
eval_logits = model(images, training=False)
```

Mã 30. Chế độ training điều khiển BN và dropout trong model.

tf.function biên dịch bước train và infer để giảm chi phí gọi Python. Lần gọi đầu thường cần tạo graph và khởi tạo kernel, nên thời gian epoch đầu có thể khác các epoch sau. reduce_retracing=True giúp hạn chế tạo lại graph khi shape thay đổi, nhưng không biến mọi chi phí khởi tạo thành 0.

Tình huống	Hành vi trong bài
Train	BN theo batch, dropout tạo mask
Validation/test	BN dùng running, dropout identity
Batch cuối nhỏ hơn 128	Vẫn được xử lý và tính trọng số theo số mẫu
Đầu ra infer	Tensor logits chuyển về NumPy để dùng hàm metric chung
Biên dịch graph	Được tính trong thời gian chạy thực tế, không tách benchmark

Bảng 48. Hành vi TensorFlow theo chế độ và shape.

Keras Adam và bản tự viết dùng cùng learning rate, betas và epsilon được khai báo. Tuy nhiên chi tiết đặt epsilon trong công thức hiệu chỉnh có thể khác, nên cùng giá trị siêu tham số không bảo đảm từng bước cập nhật bằng nhau. Kiểm tra chính của bài đối chiếu forward và gradient; không tuyên bố ba optimizer đã khớp bitwise trong mọi trạng thái.

Những khác biệt này không làm mất giá trị so sánh mô hình, nhưng giới hạn cách diễn giải chênh lệch nhỏ giữa framework. Phần nhiều seed đánh giá kết quả cuối trong điều kiện đã nêu; nếu cần cô lập riêng optimizer, phải thiết kế một phép thử cập nhật trọng số khác.

7.5. GPU Windows và checkpoint TensorFlow

```
gpus = tf.config.list_physical_devices('GPU')
for gpu in gpus:
    tf.config.experimental.set_memory_growth(gpu, True)

model.save_weights(checkpoint_path)
model.load_weights(checkpoint_path)
logits = model(images, training=False)
```

Mã 31. Cấu hình bộ nhớ trước tạo model và khôi phục weights.

Môi trường thực nghiệm dùng TensorFlow 2.10.1 với CUDA 11.2 và cuDNN 8.1. Hướng dẫn chính thức ghi giới hạn hỗ trợ GPU Windows bản địa ở nhánh 2.10 [12]. Đường dẫn DLL được cấu hình trước khi import TensorFlow; PyTorch và TensorFlow chạy trong các tiến trình riêng để quản lý thư viện GPU rõ ràng.

Memory growth cho phép TensorFlow tăng bộ nhớ theo nhu cầu thay vì đặt trước toàn bộ dung lượng có thể dùng. Thiết lập phải diễn ra trước khi runtime GPU được khởi tạo. Nó không bảo đảm tránh mọi lỗi hết bộ nhớ nếu model hoặc batch vượt tài nguyên thực tế.

File / thiết lập	Vai trò
weights.h5	Trọng số và running statistics của model
config.json	Phiên bản, kiến trúc, seed và thiết bị
training=False	Dùng chế độ suy luận sau khi khôi phục
Tiến trình CPU mới	Kiểm tra checkpoint độc lập với model còn trong bộ nhớ
environment.yml	Bộ phiên bản tương thích để tái tạo môi trường

Bảng 49. Các yếu tố để tái sử dụng checkpoint TensorFlow.

Như các backend khác, save_weights trong bài phục vụ khôi phục suy luận, không phải một snapshot đầy đủ để tiếp tục chính xác optimizer. Sau khi chọn checkpoint theo validation loss, model được nạp lại rồi chạy toàn bộ test. Sáu ảnh cố định được chạy lại trong một tiến trình CPU để đổi chiều logits đã lưu.

Hướng dẫn Anaconda phân biệt môi trường thực tế trên máy người học với môi trường sạch tạo từ YAML. Các phiên bản phân tích/Notebook có thể khác giữa hai cách cài, nhưng phiên bản thực tế luôn được ghi lại. Không sử dụng việc kernel mở được làm bằng chứng rằng toàn bộ model và checkpoint đã chạy đúng.

7.6. Đối chiếu TensorFlow và ba cách cài đặt

Dataset	Kiến trúc	Sai số logits train	Sai số gradient Conv1
mnist	baseline	1.55e-06	2.53e-07
mnist	improved	3.34e-06	4.77e-06
cifar10	baseline	2.44e-06	4.17e-07
cifar10	improved	3.70e-06	3.55e-06
cifar100	baseline	3.10e-06	4.92e-07
cifar100	improved	5.01e-06	1.59e-05

Bảng 50. Đối chiếu TensorFlow với NumPy trên batch kiểm tra CPU.

Trước khi so, gradient đầu vào TensorFlow được đổi từ NHWC về NCHW và gradient kernel từ HWIO về OIHW. Cùng các mảng không có nghĩa cùng thứ tự lưu, nên không thể so trực tiếp trước chuyển trục. Kiểm tra eval còn xác nhận BN đã giữ state tương ứng sau một forward train.

Khía cạnh	NumPy	PyTorch	TensorFlow
Ưu điểm cho bài học	Thấy rõ từng đạo hàm	Module và can thiệp linh hoạt	Graph và Functional API rõ nhánh
Trách nhiệm người viết	Toàn bộ backward	Forward và quản lý train/eval	Forward, tape và chế độ training
Rủi ro nổi bật	col2im, cache, gradient	Quên zero_grad/eval	Layout, mất kết nối tape
Minh chứng chung	Test số học	So với NumPy	So với NumPy

Bảng 51. Bài học kỹ thuật từ ba cách triển khai.

Mỗi công cụ giúp nhìn mô hình ở một mức trừu tượng khác. Bản từ đầu làm rõ cơ chế; framework giúp chạy nhanh hơn và tổ chức các thí nghiệm rộng hơn. Bài sử dụng sự hỗ trợ này để kiểm tra tính đúng và phân tích kết quả, thay vì coi một công cụ là lựa chọn tốt nhất cho mọi nhiệm vụ.

Tổng cộng mười hai cấu hình đối chiếu framework được kiểm tra, gồm hai framework, ba dataset và hai kiến trúc. Đây là kiểm chứng trước huấn luyện với dropout tắt. Các bảng accuracy nhiều seed ở chương sau là một nhóm bằng chứng khác, thu được từ học trên toàn bộ dữ liệu đã chia.

CHƯƠNG 8. KẾT QUẢ VÀ THẢO LUẬN

8.1. Cách đọc bảng nhiều seed

Tất cả 72 lượt trong giao thức đã hoàn thành đủ số epoch và tạo đầu ra test. Nhóm chính gồm 54 lượt baseline/improved; nhóm ablation gồm 18 lượt. Mỗi lượt dùng toàn bộ train và validation đã chia, đánh giá đủ 10.000 ảnh test sau khi khôi phục checkpoint có validation loss thấp nhất.

$$\bar{a} = \frac{1}{3} \sum_{s=1}^3 a_s$$

$$SD(a) = \sqrt{\frac{\sum_{s=1}^3 (a_s - \bar{a})^2}{3 - 1}}$$

Bảng ghi trung bình $\pm$ SD mẫu của ba seed, không phải sai số chuẩn hay khoảng tin cậy 95%. Accuracy và top-5 biểu diễn bằng phần trăm; chênh lệch accuracy được ghi theo điểm phần trăm. Macro-F1 được ghi trong thang 0-1. Các giá trị chưa làm tròn có trong CSV.

Nhóm	Seed	Cấu hình	Lượt
Chính	42, 7, 2026	3 dataset $\times$ 3 backend $\times$ 2 model	54
Ablation	42, 7, 2026	2 CIFAR $\times$ 3 can thiệp $\times$ PyTorch	18
Pilot riêng	42	Các lượt của phiên bản trước	4

Bảng 52. Phân nhóm kết quả; pilot không tham gia trung bình chính.

Learning curves nhiều seed vẽ trung bình theo từng epoch, phần tô biểu diễn ± 1 SD của validation. Metric test của từng seed thuộc checkpoint riêng được chọn bằng validation; trung bình checkpoint không phải một model ensemble và đường trung bình không tạo ra một epoch chung để đánh giá lại test.

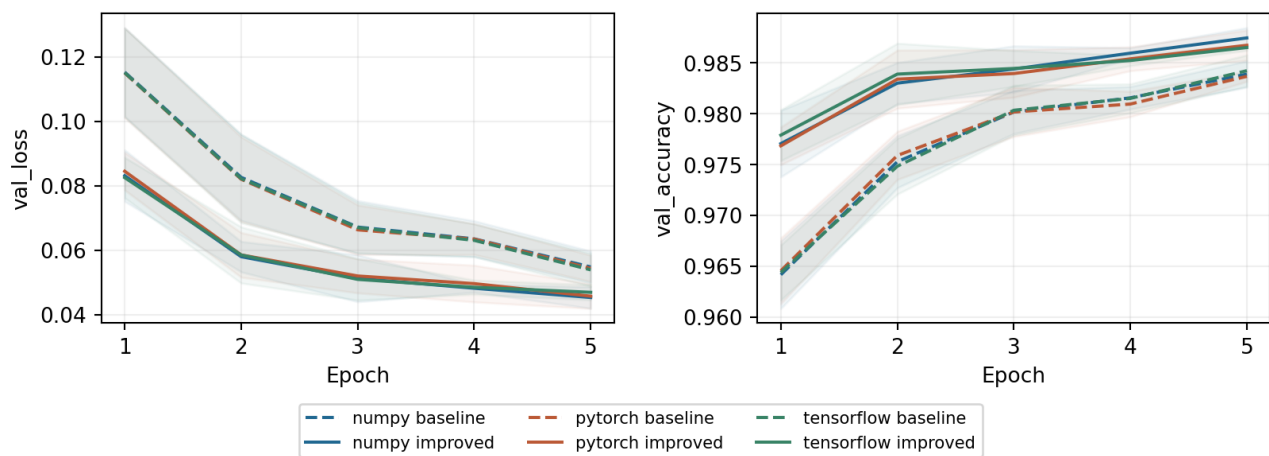
Kết quả được tính lại từ logits, đối chiếu nhãn, confidence, confusion matrix và best_epoch. Phân phân tích ảnh lỗi chọn cố định PyTorch seed 42 cho cả ba dataset để thuận tiện đối chiếu. Không chọn seed có accuracy cao nhất rồi dùng ảnh của seed đó như đại diện cho mọi lượt.

Ba seed cùng dùng một split và cùng test, nên chỉ hỗ trợ kết luận trong phạm vi này. Bài không dùng chồng lấn thanh SD như một phép kiểm định, cũng không đưa ra p-value từ ba mẫu để tạo cảm giác chắc chắn không tương xứng với thiết kế.

8.2. Kết quả nhiều seed trên MNIST

Backend / model	Accuracy (%)	Macro-F1	Top-5 (%)
numpy / baseline	$98,55 \pm 0,13$	$0,9854 \pm 0,0012$	$99,99 \pm 0,01$
numpy / improved	$98,82 \pm 0,10$	$0,9882 \pm 0,0009$	$99,99 \pm 0,02$
pytorch / baseline	$98,53 \pm 0,10$	$0,9853 \pm 0,0009$	$99,98 \pm 0,01$
pytorch / improved	$98,84 \pm 0,17$	$0,9883 \pm 0,0017$	$99,99 \pm 0,02$
tensorflow / baseline	$98,54 \pm 0,13$	$0,9853 \pm 0,0012$	$99,98 \pm 0,01$
tensorflow / improved	$98,73 \pm 0,06$	$0,9872 \pm 0,0006$	$99,98 \pm 0,02$

Bảng 53. MNIST: trung bình $\pm$ SD của ba seed, test 10.000 ảnh mỗi lượt.



Hình 8. Validation MNIST: đường trung bình và vùng ± 1 SD; nét đứt baseline, nét liền improved.

Mức tăng accuracy trung bình của improved so với baseline lần lượt là 0,27, 0,31, 0,19 điểm phần trăm ở NumPy, PyTorch và TensorFlow. Cả ba seed của từng backend đều có chênh lệch dương trong các lượt đã chạy.

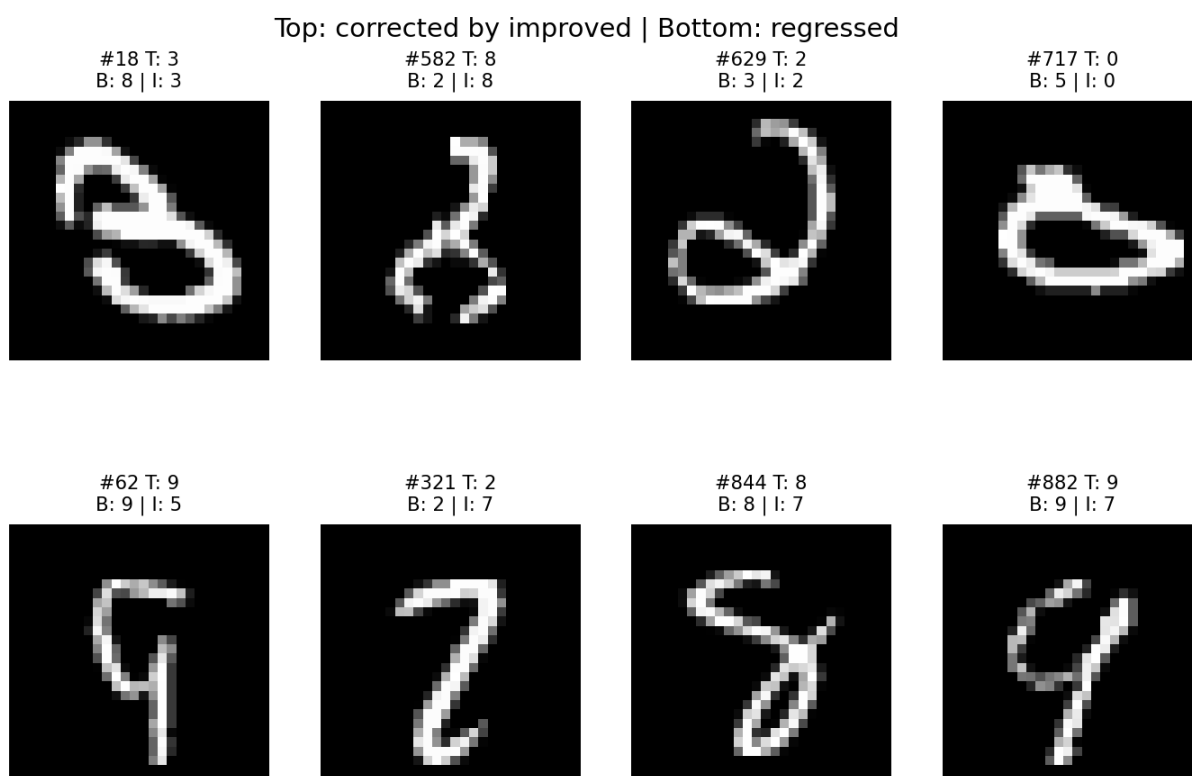
Baseline đã đạt khoảng 98,5% nên khoảng dư cải thiện nhỏ. Improved đạt trung bình khoảng 98,73-98,84%. Chênh lệch giữa framework chỉ vài phần mười điểm và dựa trên ba seed; không đủ để xếp hạng ưu thế phổ quát. Top-5 gần 100% ít phân biệt model trên bài toán chữ số này.

8.3. Phân tích lỗi MNIST

Minh họa dùng cố định PyTorch seed 42. Một ảnh có thể thuộc bốn nhóm theo việc baseline và improved dự đoán đúng hay sai. Nhóm được sửa trừ nhóm bị làm sai thêm phải khớp chênh lệch tổng số dự đoán đúng, giúp kiểm tra tính nhất quán giữa phân tích ảnh và bảng accuracy.

Cả hai đúng	Improved sửa đúng	Improved làm sai thêm	Cả hai sai
9821	82	42	55

Bảng 54. Phân hoạch 10.000 ảnh MNIST theo hai model PyTorch seed 42.



Hình 9. Hàng trên: được sửa; hàng dưới: sai thêm. T: nhãn thật, B: baseline, I: improved.

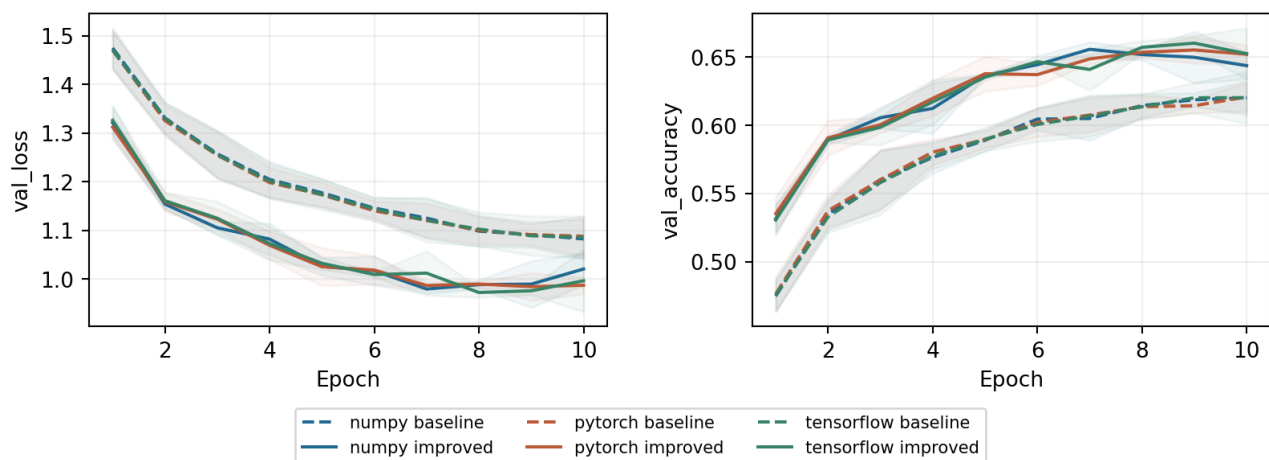
Có 82 ảnh được sửa và 42 ảnh bị làm sai thêm, nên số đúng tăng ròng 40 ảnh, tương đương 0.40 điểm phần trăm.

Nét chữ tương tự giữa các lớp khiến một thay đổi model có thể giúp một kiểu viết nhưng làm kém ở kiểu khác. Các hình được lấy theo test_id đầu tiên thỏa điều kiện, không phải một mẫu ngẫu nhiên của toàn bộ lỗi. Không suy ra nhãn dữ liệu sai chỉ từ việc model tự tin dự đoán khác.

8.4. Kết quả nhiều seed trên CIFAR-10

Backend / model	Accuracy (%)	Macro-F1	Top-5 (%)
numpy / baseline	$62,67 \pm 0,86$	$0,6239 \pm 0,0099$	$96,12 \pm 0,32$
numpy / improved	$65,21 \pm 1,06$	$0,6507 \pm 0,0111$	$96,81 \pm 0,27$
pytorch / baseline	$62,34 \pm 0,96$	$0,6210 \pm 0,0101$	$96,16 \pm 0,28$
pytorch / improved	$65,21 \pm 0,05$	$0,6514 \pm 0,0015$	$96,82 \pm 0,12$
tensorflow / baseline	$62,38 \pm 1,10$	$0,6205 \pm 0,0123$	$96,13 \pm 0,29$
tensorflow / improved	$65,56 \pm 0,66$	$0,6542 \pm 0,0071$	$96,93 \pm 0,10$

Bảng 55. CIFAR-10: trung bình $\pm$ SD của ba seed, test 10.000 ảnh mỗi lượt.



Hình 10. Validation CIFAR-10: đường trung bình và vùng ± 1 SD; nét đứt baseline, nét liền improved.

Mức tăng accuracy trung bình của improved so với baseline lần lượt là 2,54, 2,87, 3,18 điểm phần trăm ở NumPy, PyTorch và TensorFlow. Cả ba seed của từng backend đều có chênh lệch dương trong các lượt đã chạy.

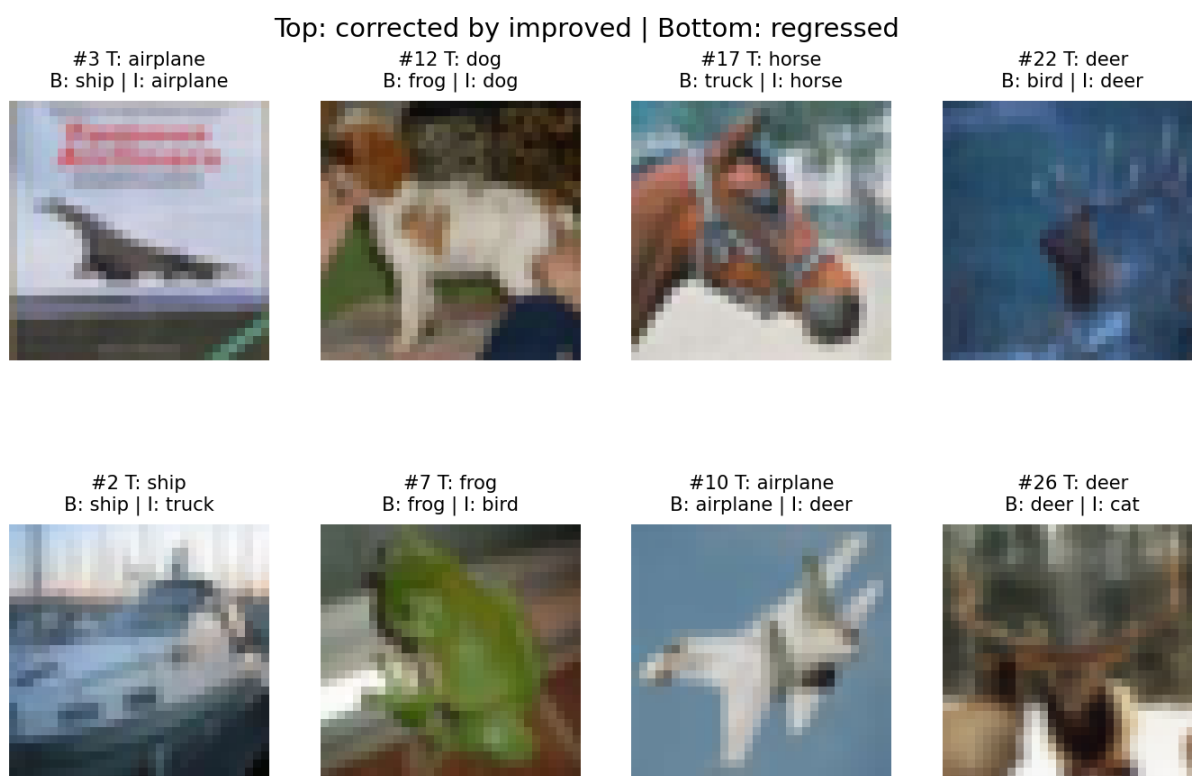
Improved đạt trung bình khoảng 65,21-65,56%, so với baseline khoảng 62,34-62,67%. Cải thiện xuất hiện ở cả ba cách triển khai, nhưng quy mô chênh lệch thay đổi theo seed. Mức top-5 cao hơn nhiều accuracy cho thấy nhân thật thường còn trong nhóm ứng viên mạnh dù chưa đứng đầu.

8.5. Phân tích lỗi CIFAR-10

Minh họa dùng cố định PyTorch seed 42. Một ảnh có thể thuộc bốn nhóm theo việc baseline và improved dự đoán đúng hay sai. Nhóm được sửa trừ nhóm bị làm sai thêm phải khớp chênh lệch tổng số dự đoán đúng, giúp kiểm tra tính nhất quán giữa phân tích ảnh và bảng accuracy.

Cả hai đúng	Improved sửa đúng	Improved làm sai thêm	Cả hai sai
5246	1280	1057	2417

Bảng 56. Phân hoạch 10.000 ảnh CIFAR-10 theo hai model PyTorch seed 42.



Hình 11. Hàng trên: được sửa; hàng dưới: sai thêm. T: nhãn thật, B: baseline, I: improved.

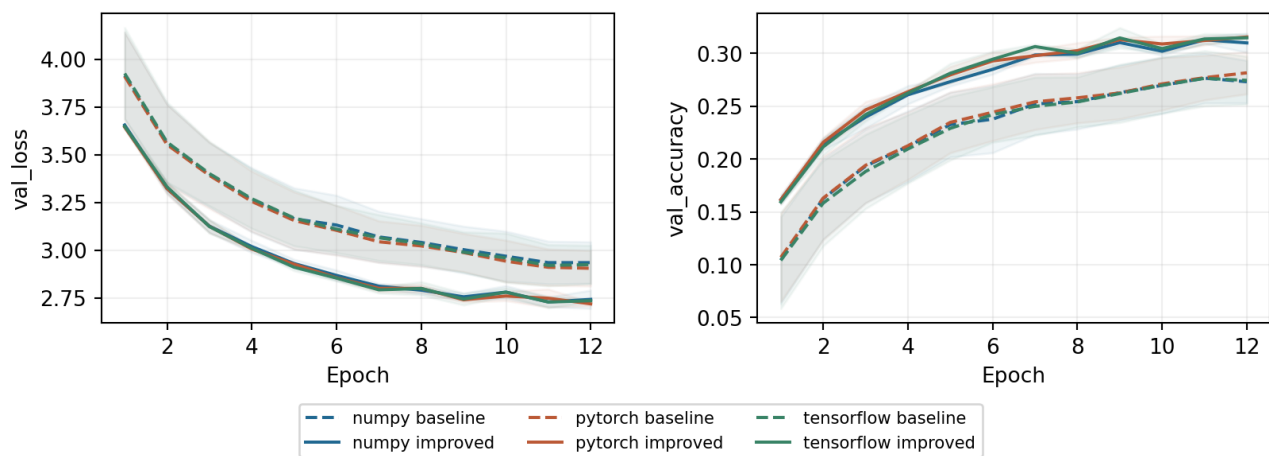
Có 1280 ảnh được sửa và 1057 ảnh bị làm sai thêm, nên số đúng tăng ròng 223 ảnh, tương đương 2.23 điểm phần trăm.

Các ảnh 32×32 chứa rất ít chi tiết, nên hình dạng, màu và bối cảnh có thể cùng ảnh hưởng. Những ví dụ này cho thấy cải thiện tổng accuracy vẫn đi kèm một nhóm ảnh bị dự đoán sai thêm. Chỉ từ hình minh họa chưa thể kết luận model luôn dựa vào nền hoặc một thuộc tính ngữ nghĩa cụ thể.

8.6. Kết quả nhiều seed trên CIFAR-100

Backend / model	Accuracy (%)	Macro-F1	Top-5 (%)
numpy / baseline	28,21 $\pm$ 2,90	0,2694 $\pm$ 0,0298	57,81 $\pm$ 3,25
numpy / improved	32,32 $\pm$ 0,16	0,3144 $\pm$ 0,0047	62,34 $\pm$ 0,67
pytorch / baseline	29,08 $\pm$ 1,91	0,2778 $\pm$ 0,0198	58,56 $\pm$ 2,74
pytorch / improved	32,41 $\pm$ 0,30	0,3145 $\pm$ 0,0055	62,44 $\pm$ 0,09
tensorflow / baseline	28,44 $\pm$ 2,20	0,2703 $\pm$ 0,0228	58,20 $\pm$ 2,96
tensorflow / improved	32,54 $\pm$ 0,14	0,3145 $\pm$ 0,0032	62,39 $\pm$ 0,67

Bảng 57. CIFAR-100: trung bình $\pm$ SD của ba seed, test 10.000 ảnh mỗi lượt.



Hình 12. Validation CIFAR-100: đường trung bình và vùng ± 1 SD; nét đứt baseline, nét liền improved.

Mức tăng accuracy trung bình của improved so với baseline lần lượt là 4,11, 3,33, 4,10 điểm phần trăm ở NumPy, PyTorch và TensorFlow. Cả ba seed của từng backend đều có chênh lệch dương trong các lượt đã chạy.

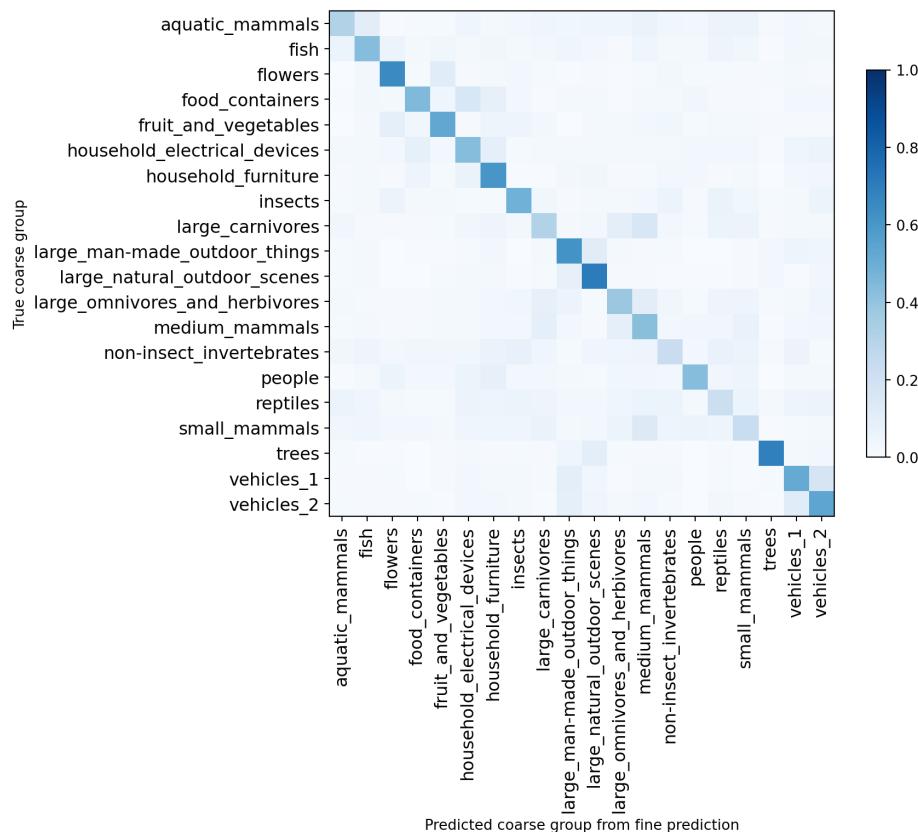
Improved đạt khoảng 32,32-32,54% và có SD accuracy khoảng 0,14-0,30 điểm, nhỏ hơn SD baseline trong ba seed đã chọn. Baseline thay đổi mạnh hơn theo khởi tạo/thứ tự batch; điều này làm SD của chênh lệch còn lớn. Mạng nhỏ và ngân sách 12 epoch vẫn hạn chế chất lượng tuyệt đối.

8.7. Phân tích lỗi CIFAR-100

Minh họa dùng cố định PyTorch seed 42. Một ảnh có thể thuộc bốn nhóm theo việc baseline và improved dự đoán đúng hay sai. Nhóm được sửa trừ nhóm bị làm sai thêm phải khớp chênh lệch tổng số dự đoán đúng, giúp kiểm tra tính nhất quán giữa phân tích ảnh và bảng accuracy.

Cả hai đúng	Improved sửa đúng	Improved làm sai thêm	Cả hai sai
2050	1165	834	5951

Bảng 58. Phân hoạch 10.000 ảnh CIFAR-100 theo hai model PyTorch seed 42.



Hình 13. Gộp nhãn fine thành 20 nhóm coarse sau dự đoán; chuẩn hóa theo nhãn thật.

Mã trận coarse giúp đọc cấu trúc lỗi của 100 lớp ở độ phân giải dễ xem hơn. Các giá trị này được suy từ dự đoán fine của cùng model, không phải accuracy của một mạng huấn luyện trên 20 lớp. Mã trận fine 100×100 và recall từng lớp vẫn có đầy đủ trong file đầu ra.

Lớp có recall thấp	Recall (%)	Số mẫu test
otter	2,00	100
seal	4,00	100
snail	5,00	100
worm	8,00	100

Bảng 59. Bốn lớp recall thấp trong model minh họa CIFAR-100.

8.8. Chênh lệch ghép seed và độ ổn định

Dataset / backend	Δ trung bình $\pm$ SD (đpt)	Seed tăng / tổng
mnist / numpy	0,27 $\pm$ 0,04	3/3
mnist / pytorch	0,31 $\pm$ 0,08	3/3
mnist / tensorflow	0,19 $\pm$ 0,13	3/3
cifar10 / numpy	2,54 $\pm$ 0,89	3/3
cifar10 / pytorch	2,87 $\pm$ 0,91	3/3
cifar10 / tensorflow	3,18 $\pm$ 1,48	3/3
cifar100 / numpy	4,11 $\pm$ 3,04	3/3
cifar100 / pytorch	3,33 $\pm$ 1,70	3/3
cifar100 / tensorflow	4,10 $\pm$ 2,33	3/3

Bảng 60. Accuracy improved trừ baseline theo từng cặp cùng seed.

Cả 27/27 cặp có improved cao hơn baseline. Đây là một mẫu kết quả nhất quán trong giao thức đã chạy, mạnh hơn nhận xét từ riêng seed 42. Tuy nhiên ba seed trên một split chưa đủ bao phủ mọi nguồn ngẫu nhiên hoặc mọi tập ảnh có thể gặp.

Ở CIFAR-100, mức tăng trung bình khoảng 3,33-4,11 điểm phần trăm nhưng SD chênh lệch từ khoảng 1,70 đến 3,04 điểm. Nhìn chỉ trung bình sẽ che việc baseline ở một số seed học tốt hơn hoặc kém hơn đáng kể. Cần đọc cả ba giá trị ở phụ lục để thấy độ biến thiên cụ thể.

$$\Delta_s = 100(a_{\text{improved},s} - a_{\text{baseline},s})$$

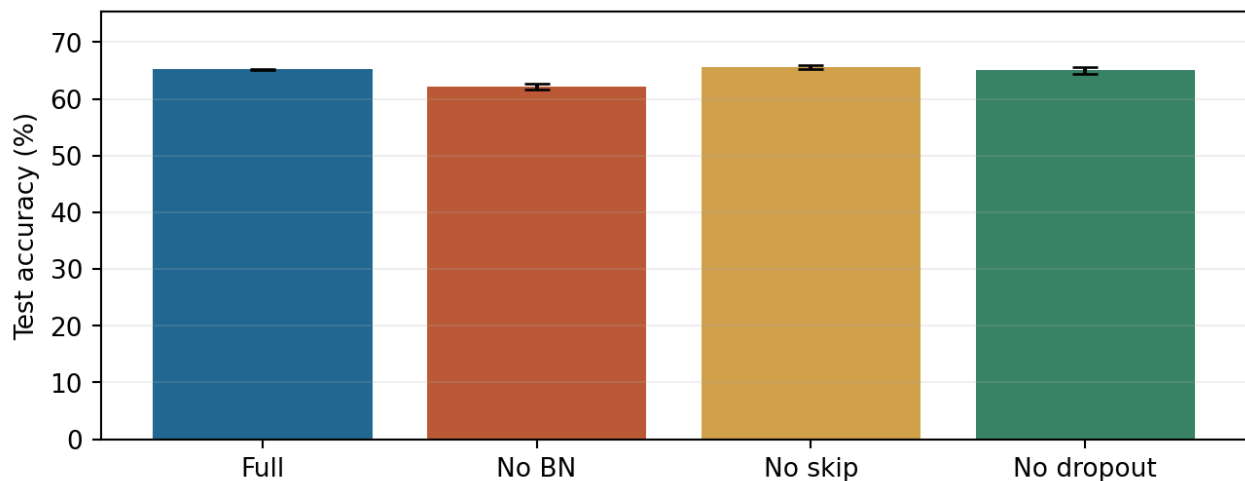
Độ lệch chuẩn của Δ được tính từ các cặp này. Công thức căn tổng hai phương sai chỉ đúng trong những giả định tương ứng về hiệp phương sai; bài không thay thế dữ liệu cặp bằng một công thức độc lập khi đã có cùng seed. Ghép seed tạo một cách đối chiếu có cấu trúc, nhưng không làm hai kiến trúc trở thành cùng một hàm tối ưu.

Không cộng ba seed thành một test gồm 30.000 ảnh độc lập: ba lần đều dự đoán cùng 10.000 ảnh. Không gọi trung bình accuracy là ensemble accuracy: không có bước trung bình logits giữa model để tạo dự đoán mới. Những phân biệt này giúp các con số được diễn giải đúng với cách tính.

8.9. Đánh giá thành phần trên CIFAR-10

Model	Accuracy (%)	F1 TB	Δ so với full (đpt)	Tham số
Full improved	$65,21 \pm 0,05$	0,6514	0,00	70170
no_bn	$62,17 \pm 0,55$	0,6195	$-3,04 \pm 0,55$	69962
no_skip	$65,61 \pm 0,38$	0,6566	$0,40 \pm 0,33$	70170
no_dropout	$64,99 \pm 0,63$	0,6467	$-0,22 \pm 0,58$	70170

Bảng 61. PyTorch: trung bình ba seed; delta = biến thể trừ full.



Hình 14. Accuracy và SD của full cùng ba biến thể tắt thành phần.

Bỏ toàn bộ BN làm accuracy trung bình giảm 3,04 điểm, và cả ba seed đều giảm. Kết quả hỗ trợ việc giữ chuẩn hóa trong kiến trúc này, nhưng chưa tách được vai trò riêng của BN ở convolution và BN sau Dense.

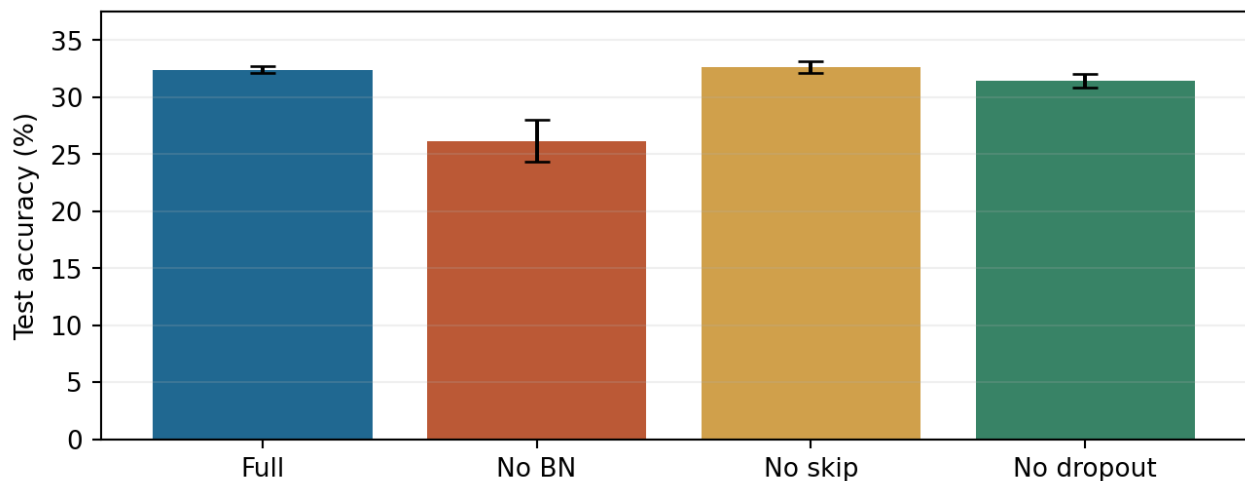
Tắt skip tăng trung bình 0,40 điểm; ba chênh lệch là +0,71, +0,44 và +0,06. Như vậy đường cộng không thể được gọi là nguồn lợi ích đã được chứng minh trên CIFAR-10 trong ngân sách hiện tại. Bỏ dropout giảm trung bình 0,22 điểm nhưng dấu thay đổi giữa seed; tác động còn nhỏ và chưa ổn định.

Đây là can thiệp từng thành phần từ full, không phải khảo sát mọi tổ hợp. Không chọn lại model theo bảng test này rồi tuyên bố nó được đánh giá độc lập trên cùng test. Kết quả cung cấp giả thuyết cho một vòng nghiên cứu mới với validation và holdout phù hợp.

8.10. Đánh giá thành phần trên CIFAR-100

Model	Accuracy (%)	F1 TB	Δ so với full (đpt)	Tham số
Full improved	$32,41 \pm 0,30$	0,3145	0,00	76020
no_bn	$26,16 \pm 1,83$	0,2422	$-6,25 \pm 1,55$	75812
no_skip	$32,65 \pm 0,50$	0,3177	$0,24 \pm 0,80$	76020
no_dropout	$31,42 \pm 0,58$	0,3062	$-0,98 \pm 0,34$	76020

Bảng 62. PyTorch: trung bình ba seed; delta = biến thể trừ full.



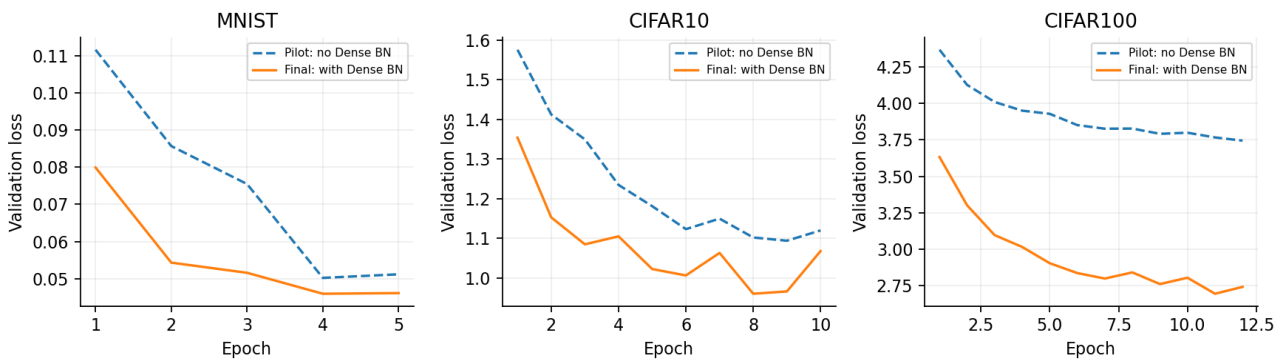
Hình 15. Accuracy và SD của full cùng ba biến thể tắt thành phần.

Bỏ BN làm giảm trung bình 6,25 điểm và giảm ở cả ba seed. Bỏ dropout giảm khoảng 0,98 điểm, cũng cùng dấu âm ở ba seed. Trong cấu hình CIFAR-100 này, hai thành phần có bằng chứng thực nghiệm rõ hơn về lợi ích đối với accuracy trong ngân sách đã chọn.

Tắt skip tăng trung bình 0,24 điểm nhưng SD chênh lệch khoảng 0,80 điểm; các seed có cả tăng và giảm. Vì vậy chưa có bằng chứng nhất quán rằng skip giúp model nhỏ này. Phát hiện đó không phủ định residual trong các kiến trúc sâu khác, vốn khác độ sâu, tối ưu và dữ liệu.

Đây là can thiệp từng thành phần từ full, không phải khảo sát mọi tổ hợp. Không chọn lại model theo bảng test này rồi tuyên bố nó được đánh giá độc lập trên cùng test. Kết quả cung cấp giả thuyết cho một vòng nghiên cứu mới với validation và holdout phù hợp.

8.11. Pilot và quá trình điều chỉnh kiến trúc



Hình 16. Pilot thiếu BN Dense và phiên bản cuối: validation loss trong TensorFlow.

Dataset	Phiên bản	Train loss cuối	Val loss tốt nhất
mnist	Thiếu BN Dense	0,0573	0,0502
mnist	Có BN Dense	0,0365	0,0459
cifar10	Thiếu BN Dense	1,1805	1,0943
cifar10	Có BN Dense	0,8290	0,9607
cifar100	Thiếu BN Dense	3,9246	3,7442
cifar100	Có BN Dense	2,5328	2,6943

Bảng 63. Các giá trị train/validation của thử nghiệm ban đầu.

Trên CIFAR-100, pilot có cả train loss và validation loss cao. Sau bổ sung BN sau Dense, hai loại loss giảm rõ trong cùng ngân sách. Mẫu đường học này phù hợp với một vấn đề tối ưu hoặc underfitting của cấu hình thử, không phải một trường hợp chỉ cần regularization mạnh hơn để giảm khoảng cách train-test.

Pilot khác ablation no_bn: pilot chỉ thiếu BN ở Dense, còn no_bn bỏ cả bốn BN của improved cuối. Hai phép đối chiếu vì vậy trả lời câu hỏi khác nhau và không được gộp số liệu. Pilot cũng chưa được lặp ba seed theo giao thức mở rộng nên không xuất hiện trong bảng mean $\pm$ SD chính.

Một số test metrics của pilot đã được tạo trước điều chỉnh; toàn bộ quy trình phát triển vì thế không mù với test. Báo cáo giữ source và bốn lượt thử cũ để công khai lịch sử, đồng thời ghi rõ quyết định điều chỉnh dựa trên dấu hiệu train/validation.

8.12. Chi phí và kiểm chứng sản phẩm

Dataset	Backend	Train baseline TB (s)	Train improved TB (s)
mnist	numpy	262,1	717,0
mnist	pytorch	26,2	52,6
mnist	tensorflow	24,8	34,5
cifar10	numpy	794,5	1583,6
cifar10	pytorch	50,6	108,7
cifar10	tensorflow	47,0	63,9
cifar100	numpy	1054,4	1836,0
cifar100	pytorch	62,0	127,9
cifar100	tensorflow	54,7	74,8

Bảng 64. Thời gian train trung bình của ba seed; không phải benchmark cô lập.

NumPy dùng CPU; PyTorch/TensorFlow dùng GPU khi có. Các hàng đợi CPU và GPU có lúc chạy đồng thời, số luồng và khởi tạo framework cũng ảnh hưởng thời gian. Do đó bảng mô tả chi phí quan sát được, không chứng minh một backend có tốc độ nội tại cao hơn theo tỷ lệ trong bảng.

Kiểm chứng	Kết quả
Metric, predictions, confidence, confusion và best epoch	72/72 lượt hợp lệ
Tải checkpoint ở tiến trình CPU mới	72/72; sáu ảnh cố định mỗi checkpoint
Notebook có đầu ra thực thi	8 notebook; 60 ô code; 0 lỗi
Unit test NumPy và ablation	15 test; 0 failure; 0 error

Bảng 65. Kiểm chứng dữ liệu kết quả, checkpoint và notebook trước đóng gói.

Metric trong PDF được làm tròn để đọc. File CSV/JSON và logits giữ độ chính xác gốc; checkpoint giữ trạng thái dùng để dự đoán. Các bằng chứng hỗ trợ này giúp phát hiện nhầm thư mục, nhầm epoch hoặc bỏ sót state, thay vì chỉ kiểm tra báo cáo có đủ biểu đồ.

8.13. Hạn chế và độ tin cậy

Phạm vi thống kê và ngân sách

Ba seed chưa đủ ước lượng chính xác toàn bộ phân phối kết quả. Split cố định không đo biến thiên khi đổi tập chia; test đã được quan sát trong quá trình phát triển. Kiến trúc nhỏ, 5-12 epoch, chưa có augmentation hoặc lịch learning rate làm chất lượng CIFAR còn hạn chế. Các kết luận cần giữ trong điều kiện đó.

Backend / model	Test gốc TB (%)	Bỏ ảnh trùng TB (%)
numpy / baseline	28,21	28,21
numpy / improved	32,32	32,32
pytorch / baseline	29,08	29,07
pytorch / improved	32,41	32,41
tensorflow / baseline	28,44	28,44
tensorflow / improved	32,54	32,54

Bảng 66. CIFAR-100: độ nhạy trung bình ba seed khi loại 10 ảnh test trùng pixel.

Phép lọc còn 9.990 ảnh và thay đổi accuracy trung bình rất nhỏ trong các lượt này. Nó không xử lý bốn hash chung train-validation, không phát hiện near-duplicate và không tạo một benchmark mới độc lập. Giữ kết quả chính thức cùng phân tích độ nhạy giúp người đọc thấy quy mô tác động thay vì che hạn chế.

Giới hạn của diễn giải model

Ablation chỉ xét từng can thiệp trên PyTorch và hai CIFAR, chưa xét tương tác đầy đủ hoặc model sâu khác. Ảnh lỗi là các trường hợp được chọn theo quy tắc, không phải mẫu ngẫu nhiên. Confidence softmax chưa được calibration; không dùng trực tiếp làm xác suất tin cậy trong quyết định thực tế.

Improved tăng accuracy ở mọi cặp đã chạy nhưng không đúng hơn baseline trên mọi ảnh. Bỏ skip có thể đạt tương đương hoặc cao hơn trong thí nghiệm này; kết luận “càng nhiều cơ chế càng tốt” không được dữ liệu hỗ trợ. Giá trị của nghiên cứu nằm ở việc giữ cả quan sát thuận và không thuận với kỳ vọng ban đầu.

CHƯƠNG 9. KẾT LUẬN

9.1. Trả lời các câu hỏi nghiên cứu

Câu hỏi	Kết luận có căn cứ trong bài
Q1. Bản NumPy có đúng không?	Các test gradient, shape, ổn định loss và học bài toán nhỏ đều đạt; CNN học trên cả ba dataset.
Q2. Ba backend có tương ứng không?	Đối chiếu logits và gradient trên 12 cấu hình đạt dung sai sau khi chuyển đúng layout và state BN.
Q3. Improved có cải thiện không?	Accuracy tăng ở cả 27 cặp cùng seed; mức tăng trung bình lớn hơn trên CIFAR so với MNIST.
Q4. Thành phần nào có bằng chứng tốt?	BN có tác động rõ ở cả hai CIFAR; dropout có lợi rõ hơn trên CIFAR-100; skip chưa giúp nhất quán.
Q5. Lỗi còn lại nói lên điều gì?	Cải thiện tổng thể vẫn có ảnh bị làm sai thêm; nhầm lớp và độ tự tin cần được đọc cùng giới hạn dữ liệu.

Bảng 67. Tổng hợp câu trả lời theo bằng chứng thực nghiệm.

Giá trị chính của bài là chuỗi kiểm chứng từ phép tính nhỏ đến thực nghiệm đầy đủ. Một công thức convolution được nối với `im2col/col2im`; một quyết định về layout được kiểm tra bằng logits; một nhận định về cải tiến được kiểm tra bằng nhiều seed và phép can thiệp. Cách trình bày này giúp người đọc truy vết kết luận về đúng dữ liệu và code.

So với bản một seed, nghiên cứu mở rộng cho thấy improved ổn định hơn baseline về accuracy CIFAR-100 trong ba seed đã chạy. Tuy nhiên kết quả ablation cũng sửa một kỳ vọng ban đầu: residual không phải thành phần có lợi nhất quán trong CNN nhỏ này. Kết luận được điều chỉnh theo dữ liệu thay vì giữ một mô tả cải tiến mặc định.

Các giới hạn vẫn còn: split và test cố định, ba seed, test đã được quan sát trong phát triển, ảnh trùng ở CIFAR-100, mạng nhỏ và ngân sách ngắn. Do đó báo cáo không tuyên bố mức tối ưu benchmark hoặc ưu thế phổ quát của framework. Các checkpoint và đầu ra cho phép tiếp tục nghiên cứu với một giao thức rõ ràng hơn.

9.2. Hướng phát triển và bài học thực hành

Hướng tiếp theo	Mục đích	Điều kiện để so sánh đúng
Split theo nhóm nội dung	Giảm ảnh trùng/near-duplicate giữa các phần	Thực hiện trước mọi tìm kiếm và đánh giá mới
Thêm seed hoặc nguồn test mới	Đánh giá rộng hơn độ biến thiên và tổng quát	Không chọn seed đẹp hoặc chỉnh theo test
Augmentation và lịch learning rate	Tăng khả năng học ảnh tự nhiên	Áp dụng cùng ngân sách cho model đối chiều
Mạng rộng hơn / global pooling	Khảo sát giới hạn biểu diễn và Dense	Đếm lại tham số, chi phí và shape
Tổ hợp ablation đầy đủ	Đánh giá tương tác BN, skip, dropout	Có giao thức mới và nguồn lực đủ
Ba dataset ban đầu	Mở rộng sang bảng và văn bản nếu được yêu cầu	Thiết kế Conv1D và baseline phù hợp từng dữ liệu

Bảng 68. Các hướng chưa thực nghiệm và mục đích cụ thể.

Khi trình bày bài, cần giải thích được một bước forward và backward nhỏ trước khi diễn giải các bảng accuracy. Việc nắm shape, cách cộng gradient và trạng thái train/eval giúp xử lý câu hỏi về code tốt hơn việc chỉ ghi nhớ số liệu. Notebook ví dụ xuyên suốt được xây dựng để hỗ trợ đúng mục đích này.

Một bài học khác là tách lỗi cài đặt khỏi giới hạn mô hình. Loss không giảm có thể do gradient sai, dữ liệu sai hoặc cấu hình tối ưu chưa phù hợp; accuracy chưa cao sau khi gradient đã được kiểm chứng có thể phản ánh độ khó dữ liệu và năng lực mạng. Các kiểm tra có cấu trúc giúp thu hẹp nguyên nhân thay vì thay siêu tham số thiếu định hướng.

Bài nộp gồm báo cáo, tám notebook có đầu ra, source, dữ liệu kết quả, checkpoint và README. Người học có thể đọc kết quả ngay hoặc chạy lại theo giao thức. Những thí nghiệm mở rộng trong tương lai cần được lưu riêng và ghi rõ khác biệt, giữ khả năng phân biệt kết quả hiện tại với kết quả mới.

TÀI LIỆU THAM KHẢO (1/2)

- [1] Slide học phần Phát triển các hệ thống thông minh. intel_sys_dev_slide_04_updated_11.9.pdf. Tài liệu do giảng viên cung cấp.
- [2] Deep Learning CNN Function Composition Tutorial. Tài liệu hướng dẫn do người học cung cấp; tham khảo phép hợp thành hàm và CNN.
- [3] François Chollet. Deep Learning with Python, 2nd edition. Manning, 2021. Các phần đánh giá/workflow (trang in 133-137, 160-165), CNN (202-205, 221-225), kiến trúc hiện đại (251-262).
- [4] oddrationale. MNIST in CSV. Bản dữ liệu Kaggle sử dụng trong bài. [Nguồn trực tuyến](#)
- [5] pankrzensiu. CIFAR-10 Python. Bản dữ liệu Kaggle sử dụng trong bài. [Nguồn trực tuyến](#)
- [6] fedesoriano. CIFAR-100. Bản dữ liệu Kaggle sử dụng trong bài. [Nguồn trực tuyến](#)
- [7] S. Ioffe và C. Szegedy. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. 2015. [Nguồn trực tuyến](#)
- [8] K. He, X. Zhang, S. Ren và J. Sun. Deep Residual Learning for Image Recognition. 2015. [Nguồn trực tuyến](#)
- [9] N. Srivastava và cộng sự. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. Journal of Machine Learning Research 15, 2014. [Nguồn trực tuyến](#)

Cách sử dụng tài liệu

Các công thức và khái niệm được diễn giải bằng lời của bài, kết hợp với ví dụ và source đã thực thi. Báo cáo không sao chép nguyên chương sách hoặc lấy số liệu công bố của model khác làm kết quả của Assignment 04. Hình thực nghiệm và bảng đều được tạo từ dữ liệu chạy của bài.

TÀI LIỆU THAM KHẢO (2/2)

- [10] D. P. Kingma và J. Ba. Adam: A Method for Stochastic Optimization. 2014. [Nguồn trực tuyến](#)
- [11] K. He và cộng sự. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. 2015. [Nguồn trực tuyến](#)
- [12] TensorFlow. Install TensorFlow with pip. Tham khảo giới hạn GPU trên Windows bản địa và tương thích môi trường. [Nguồn trực tuyến](#)
- [13] A. Krizhevsky, V. Nair và G. Hinton. CIFAR-10 and CIFAR-100 datasets. Nguồn gốc và metadata của benchmark. [Nguồn trực tuyến](#)
- [14] Y. LeCun, L. Bottou, Y. Bengio và P. Haffner. Gradient-Based Learning Applied to Document Recognition. Proceedings of the IEEE 86(11), 1998, 2278-2324. [Nguồn trực tuyến](#)
- [15] A. Krizhevsky, I. Sutskever và G. E. Hinton. ImageNet Classification with Deep Convolutional Neural Networks. NeurIPS 25, 2012. [Nguồn trực tuyến](#)
- [16] K. Simonyan và A. Zisserman. Very Deep Convolutional Networks for Large-Scale Image Recognition. 2014. [Nguồn trực tuyến](#)
- [17] PyTorch Tutorials. Automatic Differentiation with torch.autograd. Tham khảo khái niệm autograd; phiên bản thực thi của bài là 2.5.1. [Nguồn trực tuyến](#)
- [18] TensorFlow Core. Introduction to gradients and automatic differentiation. Tham khảo GradientTape; phiên bản thực thi của bài là 2.10.1. [Nguồn trực tuyến](#)

Nguồn trực tuyến được đối chiếu trong quá trình hoàn thiện tháng 09/2026. Tài liệu trực tuyến có thể phản ánh phiên bản mới hơn; config và runtime_versions.json xác định phiên bản thực tế dùng cho kết quả.

Báo cáo mẫu A3_05_dungvt.194.pdf chỉ được tham khảo về bố cục. Bìa lấy từ mẫu Google Docs do người học cung cấp. Source và kết quả của bài: https://github.com/namnguyen05-vn/PTHTTM_Assignment-04.

PHỤ LỤC A. CHẠY BẰNG ANACONDA

Trên máy đã thực hiện, có thể mở START_JUPYTER.bat và chọn kernel Python (Assignment 04). Dữ liệu ở E:/PTHTTM/ASG_04_data; runtime riêng dựa trên Anaconda ở E:/PTHTTM/ASG_04_runtime. Môi trường Anaconda cũ không bị thay đổi phiên bản thư viện trong quá trình triển khai bài.

```
conda env create -f environment.yml
conda activate assignment04
python -m pip install torch==2.5.1+cu121 --index-url https://download.pytorch.org/whl/cu121
python -m ipykernel install --user --name assignment04 ^
    --display-name "Python (Assignment 04)"
set CNN_CUDA_DIR=%CONDA_PREFIX%\Library\bin
set CNN_DATA_DIR=E:\PTHTTM\ASG_04_data
python -m src.download_data
jupyter lab
```

Mã 32. Lệnh tạo môi trường mới trong Anaconda Prompt; đổi đường dẫn dữ liệu nếu cần.

Mặc định các notebook đọc kết quả đã lưu và ghi rõ việc tái sử dụng. Notebook 02-04 có RETRAIN để chạy lại nhóm seed 42. Notebook 06 có RUN_MISSING để hoàn tất các lượt còn thiếu của giao thức mở rộng. Nếu muốn ghi đè một lượt đã có, dùng --force ở CLI sau khi sao lưu kết quả cũ.

```
python run_extended.py --plan-only
python run_extended.py
python -m src.train --backend pytorch --dataset cifar100 --variant improved --seed 7
python -m src.train --backend pytorch --dataset cifar10 --variant improved --ablation no_bn
python -m src.analyze_extended
python tools/verify_extended_checkpoints.py
python report/build_report.py
```

Mã 33. Các lệnh giao thức và tái tạo kết quả; runner bỏ qua lượt hoàn chỉnh khớp cấu hình.

Nếu Kaggle đổi cơ chế tải hoặc yêu cầu xác thực, tải thủ công ba ZIP từ các link đã nêu và đặt vào thư mục dữ liệu theo README. Source không chứa thông tin đăng nhập. Khi gặp lỗi môi trường, kiểm tra kernel Python trước khi thay model hoặc dữ liệu.

PHỤ LỤC B. NOTEBOOK VÀ TỔ CHỨC ĐẦU RA

Notebook	Vai trò
00_Discover_CNN	Khái niệm, phép tính và kiến trúc
01_Datasets	Nguồn Kaggle, split, hình và kiểm tra dữ liệu
02_CNN_From_Scratch_NumPy	Code mô hình, backward, Adam và kết quả seed 42
03_CNN_PyTorch	Module, vòng lặp và kết quả seed 42
04_CNN_TensorFlow	Keras, GradientTape và kết quả seed 42
05_Compare_Results	Phân tích chi tiết nhóm 18 lượt ban đầu
06_Multiple_Seeds_and_Ablation	Tổng hợp 72 lượt, SD, cặp seed và ablation
07_Worked_CNN_Example	Ví dụ số học xuyên suốt dùng trong PDF

Bảng 69. Thứ tự đọc tám notebook trong bài nộp.

Đường dẫn	Nội dung
results/<dataset_backend_variant>	Các lượt seed 42 gốc
results/multiseed/seed_7 hoặc seed_2026	Nhóm hai seed mới
results/ablation/seed_<s>	Các biến thể no_bn, no_skip, no_dropout
results/extended_runs.csv	Một hàng mỗi lượt trong 72 lượt
results/multiseed_summary.csv	Trung bình và SD của 18 cấu hình chính
results/ablation_summary.csv	Trung bình, SD và delta ghép seed của can thiệp
experiments/pilot_v1	Bốn pilot và source phiên bản trước

Bảng 70. Cấu trúc thư mục phân biệt nhóm thí nghiệm.

Mỗi thư mục lượt có config, history, metrics, predictions, confusion matrix, logits/labels và checkpoint. Config lưu seed và loại ablation; checkpoint phải được tải cùng cấu hình đó. Các bảng tổng hợp giữ liên kết về folder gốc để tránh trộn lượt hoặc đếm hai lần seed 42.

Bước tạo notebook đồng bộ code từ src rồi xóa đầu ra cũ; phải thực thi lại để có đầu ra mới. Công cụ execute_notebooks dừng khi gặp ô lỗi. README ghi thứ tự tạo lại bảng, kiểm tra checkpoint, notebook và PDF để các thành phần cùng một phiên bản.

PHỤ LỤC C. ACCURACY CỦA TỪNG SEED

Dataset / backend / model	Seed 42 (%)	Seed 7 (%)	Seed 2026 (%)
mnist / numpy / baseline	98,69	98,44	98,51
mnist / numpy / improved	98,92	98,73	98,81
mnist / pytorch / baseline	98,63	98,43	98,54
mnist / pytorch / improved	99,03	98,71	98,79
mnist / tensorflow / baseline	98,68	98,44	98,49
mnist / tensorflow / improved	98,75	98,77	98,66
cifar10 / numpy / baseline	63,15	63,18	61,67
cifar10 / numpy / improved	66,43	64,73	64,47
cifar10 / pytorch / baseline	63,03	62,74	61,25
cifar10 / pytorch / improved	65,26	65,20	65,16
cifar10 / tensorflow / baseline	63,01	63,02	61,11
cifar10 / tensorflow / improved	66,02	64,81	65,85
cifar100 / numpy / baseline	28,23	31,10	25,30
cifar100 / numpy / improved	32,42	32,13	32,41
cifar100 / pytorch / baseline	28,84	31,09	27,30
cifar100 / pytorch / improved	32,15	32,73	32,34
cifar100 / tensorflow / baseline	28,54	30,58	26,19
cifar100 / tensorflow / improved	32,54	32,40	32,67

Bảng 71. 54 lượt chính; accuracy của checkpoint chọn bằng validation loss.

Mỗi hàng là một cấu hình, ba cột là ba lượt độc lập về khởi tạo và thứ tự học trên cùng split. Không chọn giá trị lớn nhất làm đại diện. Bảng chương 8 lấy trung bình và SD mẫu từ ba giá trị chưa làm tròn tương ứng trong extended_runs.csv.

Trong các trường hợp CIFAR-100, baseline seed 7 cao hơn seed 2026 khá rõ ở nhiều backend, còn improved biến thiên nhỏ hơn. Đây là ví dụ cho thấy một seed riêng lẻ có thể tạo ấn tượng khác về mức lợi ích của cải tiến. Kết luận nên sử dụng toàn bộ các seed đã định trước.

Các chỉ số macro precision, macro recall, macro-F1, top-5 và cross-entropy của mọi lượt nằm trong CSV và JSON gốc. Bảng phụ lục ưu tiên accuracy theo seed để có thể kiểm tra trực tiếp các phép tính chênh lệch trong báo cáo.

PHỤ LỤC D. ABLATION VÀ KIỂM TRA TÁI LẬP

Dataset / can thiệp	Seed	Accuracy (%)	Macro-F1	Epoch tốt
cifar10 / no_bn	7	62,79	0,6248	10
cifar10 / no_bn	42	61,98	0,6179	10
cifar10 / no_bn	2026	61,74	0,6159	10
cifar10 / no_dropout	7	65,05	0,6457	6
cifar10 / no_dropout	42	65,59	0,6562	9
cifar10 / no_dropout	2026	64,33	0,6384	7
cifar10 / no_skip	7	65,64	0,6580	9
cifar10 / no_skip	42	65,97	0,6589	9
cifar10 / no_skip	2026	65,22	0,6528	10
cifar100 / no_bn	7	28,26	0,2616	11
cifar100 / no_bn	42	24,90	0,2314	12
cifar100 / no_bn	2026	25,31	0,2336	11
cifar100 / no_dropout	7	31,92	0,3135	7
cifar100 / no_dropout	42	30,78	0,2967	6
cifar100 / no_dropout	2026	31,57	0,3084	7
cifar100 / no_skip	7	32,08	0,3089	12
cifar100 / no_skip	42	33,03	0,3235	12
cifar100 / no_skip	2026	32,84	0,3205	12

Bảng 72. 18 lượt ablation; không đưa pilot vào bảng này.

Full improved tham chiếu là các lượt PyTorch cùng dataset và seed ở phụ lục C. Lấy accuracy của hàng ablation trừ giá trị full tương ứng tạo delta của từng seed. Trọng số còn tồn tại được khởi tạo giống full, nhưng mỗi lượt được học lại từ đầu.

```
python tools/run_test_suite.py
python -m tests.verify_framework --backend pytorch
python -m tests.verify_framework --backend tensorflow
python -m src.analyze_extended
python tools/verify_extended_checkpoints.py
python tools/execute_notebooks.py
```

Mã 34. Các lệnh kiểm tra; phiên bản chạy và đầu ra được lưu trong results.

Các file verification ghi phạm vi đã kiểm tra, không phải chứng minh hình thức cho mọi đầu vào. Khi thay source hoặc kết quả, cần chạy lại các kiểm tra liên quan và tái tạo báo cáo. Mã và dữ liệu đầu ra đầy đủ:

https://github.com/namnguyen05-vn/PTHTTM_Assignment-04.